

基于几何分析与贪心算法的烟幕干扰弹投放策略

摘要

烟幕干扰弹在特定空域起爆形成球状烟幕云团，由无人机定点投放、定时起爆以干扰敌方导弹探测真目标；对于不同的无人机数目、烟幕弹数和导弹数，本文建立了多种策略模型，用于求解烟幕干扰弹的投放策略。

对于问题一，本文首先将烟幕弹的运动过程划分为无人机携带阶段、平抛阶段和云团下沉阶段，**得到烟幕弹的运动方程**；然后分析了导弹的运动特点，得到了**各导弹的运动方程**；并分析得到了**产生有效遮蔽的条件**为：导弹与真目标的连线段与烟幕云团有效遮蔽范围相交；最后结合烟幕弹与导弹的位置信息及有效遮蔽条件，解得 FY1 干扰 M1 的有效遮蔽时长为 1.41s。

对于问题二，首先对四种有效遮蔽情况进行了几何分析，得到了每枚烟幕弹的理论最大干扰时间约为 6.7s；同时得出“烟幕云团应尽量远离真目标”的结论，**减少了需要优化的参数**，并将模型转化为单目标优化问题。基于此得到投弹策略：应将无人机 FY1 飞行速度与烟幕弹投放时间取最小值。最终通过改良网格搜索算法得到**最大遮蔽时长为 4.680s**。

对于问题三，本文将“单无人机—多烟幕弹—单导弹”的大规模优化问题拆分为了三个带有投弹时间约束的“单无人机—单烟幕弹—单导弹”的单目标优化问题。为了提高计算效率、保证解的质量，**将问题二的求解结果作为无人机的飞行参数与第一枚烟幕弹的投弹参数**，剩余两枚烟幕弹的参数则基于贪心思想递推求解。最终解得：三枚烟幕弹的最大遮蔽时长分别为 4.680s、2.88s 和 0s，总遮蔽时长为 7.56s，并将具体结果存入了 result1.xlsx 文件。

对于问题四，本文将“多无人机—单烟幕弹—单导弹”的大规模优化问题拆解为三个独立的“单无人机—单烟幕弹—单导弹”单目标优化问题：**FY1 直接复用问题二的投弹策略结果**；求解 FY2、FY3 时，先设定方位角初始猜测值，通过遗传算法完成粗略搜索，再在粗略可行解附近开展局部网格细搜。解得 **FY1、FY2、FY3 的有效遮蔽时长分别为 4.68s、4.52s、4.71s**，总遮蔽时长为 13.91s，具体结果已存入 result2.xlsx 文件。

对于问题五，将“多无人机—多烟幕弹—多导弹”的大规模优化问题拆解为独立的 15 个带有投弹时间约束的“单无人机—单烟幕弹—单导弹”的子问题。求解时，利用问题三的结果作为 FY1 的策略；利用问题四的结果作为 FY2、FY3 策略的一部分解，同时利用给定猜测方位角的遗传算法求得 FY4、FY5 的一部分解，然后用有投弹时间约束的问题二模型对四架无人机完全求解。**解得共投出 15 枚烟幕弹，有 10 枚烟幕弹有效，各烟幕弹有效遮蔽时长之和为 42.20s**，并将具体求解结果存入了 result3.xlsx 文件。

最后，对优化参数飞行方位角的敏感性进行分析，验证了问题五所拟定的投弹策略的正确性与必要性。

关键字： 烟幕干扰弹投放策略 几何分析 贪心思想 优化模型拆解 改良网格搜索

一、问题重述

1.1 问题背景

烟幕干扰弹（以下简称“烟幕弹”）在爆炸或燃烧后，可以在特定空域产生烟幕或气溶胶云团，从而形成烟幕干扰敌方导弹。烟幕弹可以由无人机进行定点精确抛撒，并通过时间引信时序定时起爆。无人机通过在来袭武器和保护目标之间投递烟幕弹产生烟幕遮蔽，进而起到保护目标的作用。

1.2 问题要求

以假目标为原点，现有来袭导弹以 300 m/s 的速度直线飞向用于掩护真目标的假目标，导弹的位置已知。要保护的真正目标为底面圆半径 7 m 、高度 10 m 的圆柱体，其位置已知。

无人机在受领掩护指令后，可以瞬时调整方向，以 $70 \sim 140\text{ m/s}$ 的速度，做水平匀速直线运动，前去投放烟幕弹，每两次投弹间隔至少 1 s 。各无人机航向与速度可不同，但在确定后不可更改。各无人机的初始位置已知。

烟幕弹脱离无人机后，在重力作用下做平抛运动；爆炸后，瞬间产生球状烟幕云团，云团中心半径 10 m 内的烟幕在起爆 20 s 内可以提供有效遮蔽；起爆后，云团以 3 m/s 的速度匀速下沉。

要求确定合适的投弹策略，使得各烟幕弹的有效遮蔽时间最长

问题 1：仅考虑无人机 FY1 对 M1 投放 1 枚烟幕弹进行干扰，FY1 以 120 m/s 向假目标方向飞行，开始飞行 1.5 s 后投放 1 枚烟幕干扰弹，投放 3.6 s 后起爆。要求计算烟幕干扰弹对 M1 的有效遮蔽时长。

问题 2：仅考虑无人机 FY1 对 M1 投放 1 枚烟幕弹进行干扰，要求确定合适的投弹策略即飞行方向、飞行速度、投弹点和起爆点，使得遮蔽的时间最长。

问题 3：考虑 FY1 投放三枚烟幕干扰弹对导弹 M1 进行干扰，要求确定 FY1 的投弹策略。

问题 4 要求利用 FY1、FY2 和 FY3 三架无人机分别投放 1 枚烟幕弹对 M1 进行干扰，要求确定三架无人机的投弹策略。

问题 5 要求利用五架无人机各自投放至多 3 枚烟幕弹对三枚导弹 M1、M2 和 M3 进行干扰，要求确定五架无人机的投弹策略。

二、问题分析

2.1 问题一分析

问题一要求在已知无人机 FY1 的初始位置、飞行速度、飞行方向、烟幕干扰弹投放时间和起爆时间的条件下，求解烟幕干扰弹对 M1 的有效遮蔽时长，本质是求解给定参数下的空间运动问题。

第一步，可根据烟幕弹的运动特点将其运动分为不同阶段，确定每个阶段烟幕弹的运动情况，进而确定烟幕弹的位置矢量关于时间 t 的函数。**第二步**，根据敌方导弹 M1 的运动情况，确定 M1 的位置矢量关于时间 t 的函数。**第三步**，根据起爆点坐标建立云团表面关于时间 t 的曲面簇，判断曲面簇是否和 M1 与真目标之间的连线段相交，相交则为有效遮蔽，进一步根据遮蔽有效的判断条件即可求解出有效遮蔽时长。

2.2 问题二分析

在问题二中，对于任意给定的 FY1 飞行速度、飞行方向、烟幕弹投放时间和起放时间，可以在问题一的模型基础上求解有效遮挡时间。可以将该给值求解的过程定义成函数，则可将问题二转化为求解该函数最大值的优化问题。

各变量中，飞行方向可以由飞行方位角表示，则需要优化 4 个变量：无人机飞行速度、飞行方位角、烟幕弹投放时间和起爆时间，使得有效遮挡时间取最大值。考虑到优化 4 个变量的计算量较大，可以尝试**减少需要优化的变量数**。根据烟幕云团不同有效遮挡情况，分析得到云团中心到原点的距离与有效遮蔽时长的关系，得出无人机飞行速度和烟幕弹投放时间也应当尽可能小，可取二者的最小值。从而仅需要考虑无人机飞行方位角与起爆时间。

针对无人机飞行方位角与起爆时间进行优化时，可以尝试基于改良后的网格搜索法搜索全局最优解。对于每个飞行方位角，可以取烟幕弹起爆后恰好距离线段 M1O 距离为球体半径时的起爆时间为猜测最优时间，并在猜测最优时间附近进行局部搜索得到局部最优解，对所有的飞行方向角进行搜索，则可得到全局最优解。

2.3 问题三分析

问题三的本质是一个多目标优化问题，可以考虑利用贪心思想，将其转化为单目标优化问题：在已知前一个投弹信息时，递推出下一个使得有效遮蔽时间最优的投弹信息。

还需要先定义总的有效遮挡时间应当是每个烟幕弹的遮蔽时间区间的测度之和。由贪心思想，可以问题二优化求解的结果作为第一个烟幕弹的投弹信息，优化下一颗烟幕弹的投弹参数，使得总有效遮蔽时间最大，进一步地可以递推出第三颗烟幕弹的投弹信息。即可求解该问题。

2.4 问题四分析

问题四的本质是包含了更多参数的多目标优化问题，若直接使用各类智能优化算法，不仅会面临极长的搜索时间，还极易陷入局部最优解，故不可取。但是，由有效遮蔽时间的定义分析可知，每个无人机对应的最大遮蔽时间是彼此独立的。因此可分别对每个无人机的运动参数与投弹参数进行优化，即分别求解无人机 FY1、FY2、FY3 干扰 M1 时的“单无人机-单烟幕弹-单导弹问题”，该过程可在问题二的模型基础上进行求解。问题二已求解出 FY1 的最大遮蔽时间，只需将问题二模型中的无人机分别替换为 FY2 和 FY3，即可求解出其对应的运动参数与投弹参数及其最大遮蔽时间。

2.5 问题五分析

对于问题五，其本质仍然为多参数多目标优化问题，需优化的参数量高达 40 个，因此直接优化的方式仍然不可取。

与问题四的分析逻辑一致，仍可依据各无人机遮蔽时间的独立性，对全局优化模型进行拆解，将其拆解为 5 个“单无人机投放策略”子问题，并进一步将每个子问题拆分为“单无人机—多烟幕弹—单导弹问题”或“单无人机—多烟幕弹—多导弹问题”的小问题(可转化为有时间约束的“单无人机—单烟幕弹—单导弹问题”)，并通过问题二和问题三的模型进行求解。

可考虑采用以下投弹策略：每个无人机均投放 3 枚烟幕弹，可根据无人机附近最近的导弹将其处理为不同的小问题，并确定各自的飞行参数与某一枚烟幕弹的投弹参数；再由此优化第二枚，第三枚烟幕弹的投弹参数，从而得到所有无人机的投弹策略。

三、模型假设

为简化问题，本文做出以下假设：

- 假设 1：无人机和起爆前的烟幕弹均视为质点。起爆后的烟幕云团视为规则球体。
- 假设 2：烟幕云团仅做匀速下沉的运动。
- 假设 3：真目标暴露在导弹的视线中当且仅当真目标底面圆圆心暴露在导弹的视线中
- 假设 4：各导弹之间不进行信息交流，若每枚导弹到真目标的视野被遮蔽，则认为该导弹丢失了真目标的视野，遮蔽有效。

四、符号说明

符号	说明	单位
$\mathbf{r}_*(t)$	运动对象 * 的位置向量函数	\
$\mathbf{v}_*(t)$	运动对象 * 的速度向量函数	\
$D_{i,j}$	第 i 架无人机投出的第 j 枚烟幕弹	\
$t_{D_{i,j},\text{Mk}}$	第 i 架无人机投出的第 j 枚烟幕弹干扰 Mk 的有效遮蔽时间	s
α_{FY_i}	无人机 FY_i 的飞行方位角	rad
$t_{D_{i,j},\text{thr}}$	第 i 架无人机投出的第 j 枚烟幕弹的投弹时间	s
$t_{D_{i,j},\text{bst}}$	第 i 架无人机投出的第 j 枚烟幕弹的起爆时间	s
$m(\cdot)$	表示可测集 $\cdot$ 的测度	\

五、问题一模型的建立和求解

5.1 模型准备

要求解本题，需要定义“单无人机—单烟幕弹—单导弹有效遮蔽时长”。

定义 1 (单无人机—单烟幕弹—单导弹有效遮蔽时长). 本文定义为对于某一烟幕云团而言，导弹与真目标底面圆圆心的线段与烟幕云团有效遮蔽范围相交的时长。

5.2 模型建立

题目一要求计算烟幕干扰弹 (本文简称“烟幕弹”) 对导弹 M1 的有效遮蔽时长，若以半径 7m、高 10m 的圆柱体作为要掩护遮蔽的对象，则需要考虑将该圆柱体向烟幕干扰弹所在方向进行二维投影，并计算投影是否被烟幕云团所遮蔽，计算量较大。考虑用真目标底面圆圆心处一点来代替圆柱体真目标，作为烟幕云团要遮蔽的对象，现说明其合理性。

首先，无人机受领任务时，敌方导弹距离真目标的**初始距离**相较于真目标的几何尺寸大了至少两个数量级；其次，各导弹均以假目标作为目的，而**假目标距离**真目标

200 m，相较于真目标几何尺寸同样大了约两个数量级；再者，利用烟幕云团遮蔽真目标时，由视觉感知近大远小的特性，在导弹的视野中，真目标相比于烟幕云团会显得更小。综上，用真目标底部圆心 A 点来替代圆柱体真目标是合理的。

记无人机 FY_i 投放的第 j 枚烟幕干扰弹为 $D_{i,j}$ (后简称烟幕弹 “ $D_{i,j}$ ” 或 “ $D_{i,j}$ ”), 要使烟幕云团对导弹 M1 产生有效遮蔽，即 M1 为与真目标底面圆圆心 A (后简称 “真目标 A” 或 “A”) 的射线 M1A 要穿过烟幕的有效遮蔽区，而烟幕的有效遮蔽区为以云团中心为球心、半径 10 m 的球体。因此，要求得有效遮蔽时长，就要求得导弹和烟幕云团的位置信息，然后判断导弹和烟幕云团的几何关系。

5.2.1 烟幕弹及导弹运动模型的建立

对于烟幕弹及云团中心，将云团中心的运动视为烟幕弹运动的一部分，则任意烟幕弹的运动可分为三个阶段：无人机携带阶段、抛物阶段和云团下沉阶段，如图 1 所示。

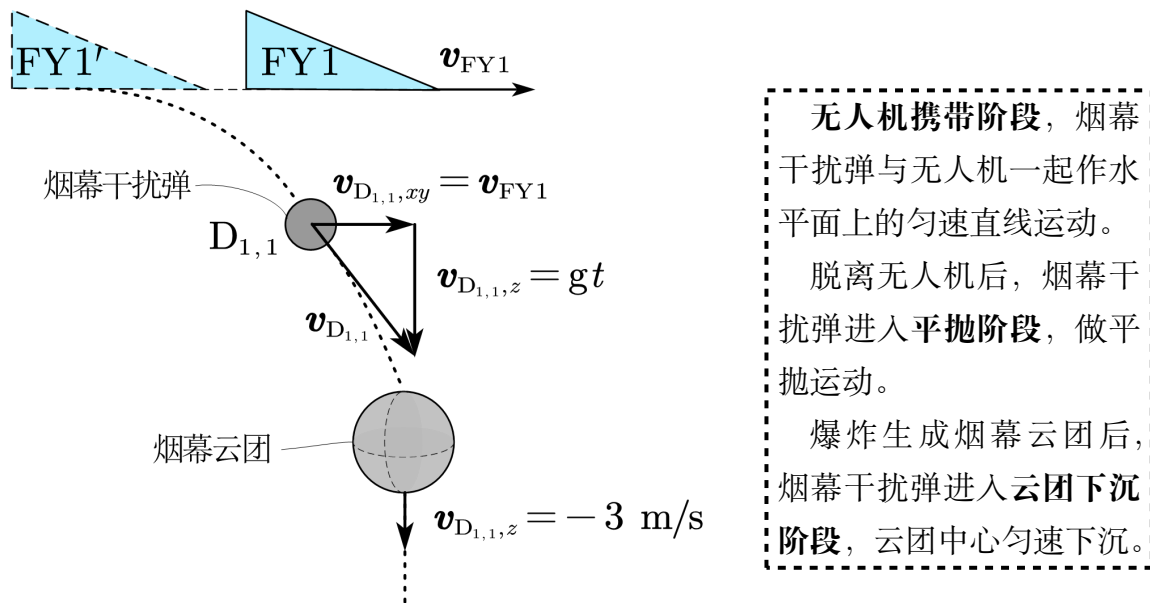


图 1 烟幕干扰弹飞行过程示意图

在无人机携带阶段，烟幕弹的位置与其所在无人机的位置重合，则只需求出无人机的位置即可表示烟幕弹的位置。令无人机 FY_i 的速度大小为 v_{FY_i} ，其方向的单位向量为 s_{FY_i} ，则 FY_i 的速度为 $v_{FY_i}s_{FY_i}$ ，令所有无人机受领任务的时刻为 0 时刻，令 0 时刻 FY_i 位置为 $P_{FY_i,0}$ ，则 t s 后，则烟幕弹 $D_{i,j}$ 的位置矢量等于无人机 FY_i 的位置矢量为：

$$r_{D_{i,j}}(t) = r_{FY_i}(t) = \mathbf{OP}_{FY_i,0} + tv_{FY_i}s_{FY_i}$$

在抛物阶段，烟幕弹被无人机抛出，然后开始做有水平初速度的平抛运动。令烟幕弹 $D_{i,j}$ 在无人机飞行 $t_{D_{1,1},thr}$ 后被投放 (后称 “投放时间”), 此时 FY_i 的位置为 $r_{FY_i}(t_{D_{1,1},thr})$ 。

令重力加速度为 g ，则烟幕弹 $D_{i,j}$ 的速度可以表示为：

$$\mathbf{v}_{D_{i,j}}(t) = v_{FYi} \mathbf{s}_{FYi} + \begin{bmatrix} 0 & 0 & -g(t - t_{D_{1,1},thr}) \end{bmatrix}^T$$

其位置矢量可以表示为：

$$\mathbf{r}_{D_{i,j}}(t) = \int_{t_{D_{i,j},thr}}^t \mathbf{v}_{D_{i,j}}(u) du + \mathbf{r}_{FYi}(t_{D_{i,j},thr}) \triangleq \begin{bmatrix} x_{D_{i,j}}(t) & y_{D_{i,j}}(t) & z_{D_{i,j}}(t) \end{bmatrix}^T$$

考虑到烟幕弹有可能坠地，有：

$$\mathbf{r}_{D_{i,j}}(t) = \int_{t_{D_{i,j},thr}}^t \mathbf{v}_{D_{i,j}}(u) du + \mathbf{r}_{FYi}(t_{D_{i,j},thr}) \quad , t < z_{D_{i,j}}^{-1}(0) \text{ (后省略)}$$

在云团下沉阶段，烟幕弹爆炸形成烟幕云团，云团以 3 m/s 的速度匀速下沉，水平方向速度分量为 0。令烟幕弹于抛出 $t_{D_{1,1},bst}$ 后爆炸 (后称“起爆时间”)，则云团中心的位置矢量可以表示为：

$$\mathbf{r}_{D_{i,j}}(t) = \mathbf{r}(t_{D_{i,j},bst}) + \begin{bmatrix} 0 & 0 & -3(t - t_{D_{1,1},thr} - t_{D_{1,1},bst}) \end{bmatrix}^T \quad , t < z_{D_{i,j}}^{-1}(0) \text{ (后省略)}$$

则在烟幕弹 $D_{i,j}$ 的整个运动过程中，其位置矢量关于时间 t 的函数为：

$$\mathbf{r}_{D_{i,j}}(t) = \begin{cases} \mathbf{OP}_{FY,0} + tv_{FYi} \mathbf{s}_{FYi} & 0 \leq t \leq t_{D_{1,1},thr} \\ \int_{t_{D_{i,j},thr}}^t \mathbf{v}_{D_{i,j}}(u) du + \mathbf{r}(t_{D_{1,1},thr}) & t_{D_{1,1},thr} \leq t \leq t_{D_{1,1},thr} + t_{D_{1,1},bst} \\ \mathbf{r}_{D_{i,j}}(t_{D_{i,j},thr} + t_{D_{i,j},bst}) + \\ \quad \begin{bmatrix} 0, 0, -3(t - t_{D_{i,j},thr} - t_{D_{i,j},bst}) \end{bmatrix}^T & t_{D_{1,1},thr} + t_{D_{1,1},bst} \leq t \end{cases} \quad (1)$$

对于任意敌方导弹 M ，其速度大小均为 300 m/s，令其方向的单位向量为 $\mathbf{s}_M$ ，无人机组受领任务时该导弹的位置为 $\overrightarrow{OP}_{M,0}$ ，则 t 时刻， M 的位置矢量为

$$\mathbf{r}_M(t) = \overrightarrow{OP}_{M,0} + 300t\mathbf{s}_M \quad t \geq 0 \quad (2)$$

5.2.2 烟幕遮蔽有效模型的建立

要使烟幕弹的遮蔽有效，则需要导弹到真目标的视线被烟幕云团的有效遮蔽区所阻挡，即从导弹到真目标的线段需要穿过云团中心半径 10 m 的球体。

由前文建立的烟幕弹及导弹运动模型，我们可以得到云团中心和导弹的位置信息，现给出判断空间中任一线段是否穿过球体的模型如图2所示。

以 O 为原点，对于空间中两点 $M(x_M, y_M, z_M)$ 和 $N(x_N, y_N, z_N)$ ，二者的连线段上的任意点 P 满足：

$$\mathbf{OP}(u) = u\mathbf{OM} + (1 - u)\mathbf{ON} \quad 0 \leq u \leq 1$$

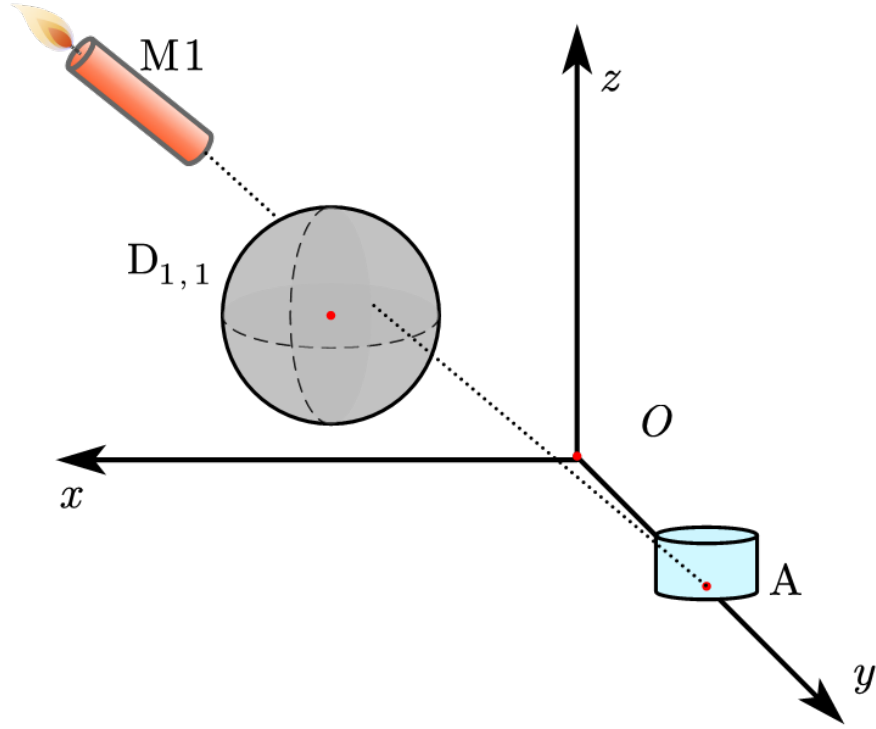


图2 烟幕云团有效遮挡示意图

对于空间中半径为 R 、球心坐标 $S(x_S, y_S, z_S)$ 的球体，若线段 MN 与其相交，则有球心 S 到线段 MN 的距离 $D_{S \rightarrow MN}$ 小于球半径，有：

$$D_{S \rightarrow MN}(t) = \min_{u \in [0,1]} \|\mathbf{OP}(u) - \mathbf{OS}\| \leq R$$

对于任意导弹 M ，令 $\mathbf{r}_M(t)$ 表示 M 位置矢量对时间 t 的函数，则 M 与真目标 A 之间的连线段可以表示为：

$$\mathbf{L}_{AM}(u, t) = u\mathbf{r}_M(t) + (1 - u)\mathbf{OA} \quad u \in [0, 1] \quad (3)$$

则烟幕弹 $D_{i,j}$ 对导弹 M 形成有效遮蔽的条件是：

$$d_{D_{i,j}, M}(t) = \min_{u \in [0,1]} \|\mathbf{L}_{AM}(u, t) - \mathbf{r}_M(t)\| \leq R \quad t > t_{D_{i,j}, \text{thr}} + t_{D_{i,j}, \text{bst}} \quad (4)$$

5.2.3 有效遮蔽时间计算模型的建立

由式(4)，可得烟幕弹 $D_{i,j}$ 对导弹 M 形成有效遮蔽的条件，定义任意烟幕弹 $D_{i,j}$ 对任意导弹 M 的有效遮蔽值：

$$E_{D_{i,j}, M} = \begin{cases} 1 & d_{D_{i,j}, M}(t) \leq R \\ 0 & \text{else} \end{cases} \quad (5)$$

则可得到 D 对 M 的有效遮蔽时间:

$$t_{D_{i,j},M} = \int_{t_{D_{i,j},bst}}^{t_{D_{i,j},bst}+20} E_{D_{i,j},M}(t) dt \quad (6)$$

5.3 模型求解

Step1: 计算烟幕弹的位置信息

考虑无人机 FY1 对 M1 的干扰, 则由式(1)得无人机 FY1 投下的第一颗烟幕弹 $D_{1,1}$ 的位置矢量方程为:

$$\mathbf{r}_{1,0}(t) = \begin{cases} \mathbf{OP}_{FY1,0} + 120t \cdot \mathbf{s}_{FY} & 0 \leq t \leq 1.5 \\ \int_{1.5}^t \mathbf{v}_{D_{1,1}}(u) du + \mathbf{r}_1(1.5) & 1.5 \leq t \leq 5.1 \\ \mathbf{r}_1(5.1) + [0, 0, -3(t-5.1)]^T & t \geq 5.1 \end{cases} \quad (7)$$

其中 $\mathbf{P}_{FY1,0} = [17800, 0, 1800]^T$, $\mathbf{s}_{FY} = [-1, 0, 0]^T$ 。

Step2: 计算导弹 M1 的位置

由式(2)得 M1 的位置矢量:

$$\mathbf{r}_{M1}(t) = \mathbf{P}_{M1,0} + 300t \cdot \mathbf{s}_{M1} \quad t \geq 0$$

其中 $\mathbf{P}_{M1,0} = [20000, 0, 2000]^T$, $\mathbf{s}_{M1} = \mathbf{PO}/\|\mathbf{PO}\|$ 。

Step3: 判断是否形成有效遮蔽

由式(4)得烟幕弹 $D_{1,1}$ 对导弹 M1 形成有效遮蔽的条件是:

$$d_{D_{1,1},M1}(t) = \min_{u \in [0,1]} \|\mathbf{L}_{AM1}(u, t) - \mathbf{r}_{1,0}(t)\| \leq R \quad t > 5.1$$

其中真目标 A 满足 $\mathbf{OA} = [0, 200, 0]^T$ 。

Step4: 计算有效遮蔽时长

利用式(6), 得烟幕干扰弹 $D_{1,1}$ 对 M1 的有效遮蔽时长为:

$$t_{D_{1,1},M1} = \int_{5.1}^{25.1} E_{D_{1,1},M1}(t) dt$$

由于烟幕弹 $D_{1,1}$ 在 $t_{D_{1,1},thr} + t_{D_{1,1},bst} = 5.1s$ 后开始爆炸, 所以可以从 5.1s 开始对烟幕干扰弹投放过程进行仿真计算, 每一时间步中重复 Step1、Step2 和 Step3, 进而实现 Step4。

5.4 求解结果

取仿真步长为 0.01s, 则由模拟仿真计算得到烟幕干扰弹的投放坐标为 $[17620, 0, 1800]^T$, 烟幕干扰弹的起爆点坐标为 $[17188, 0, 1736.50]^T$, 遮蔽时间段: 8.04 秒 - 9.45 秒, 烟幕干扰弹 $D_{1,1}$ 对 M_1 的有效遮蔽时长 $t_{D_{1,1},M1} = 1.41s$ 。

5.4.1 结果分析

遮蔽开始与结束时刻的情况如图 3 所示，如图 3a 所示，导弹与真目标之间的连线，恰与烟幕云团边缘相切，此时烟幕云团恰好将真目标遮蔽；如图 3b 所示，导弹恰好穿过云团，烟幕云团不在导弹与真目标的连线上，此时恰好不能遮蔽。

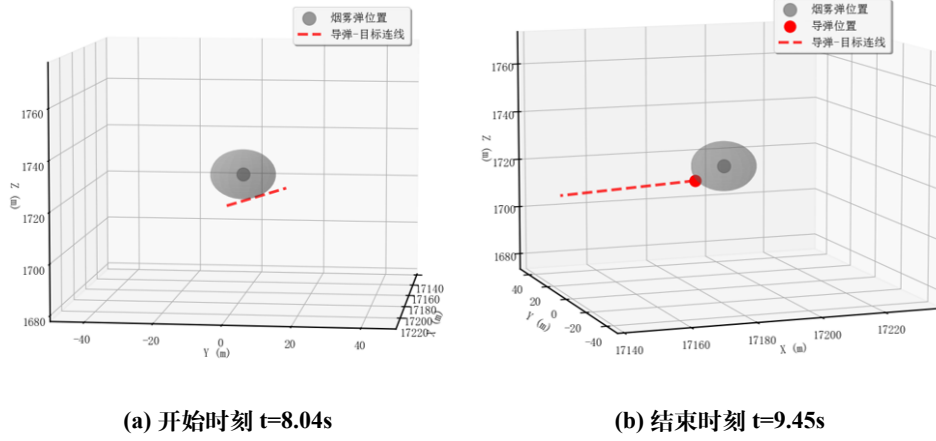


图 3 遮蔽始末烟幕弹遮蔽情况

六、问题二的模型的建立和求解

6.1 模型准备

烟幕弹 $D_{i,j}$ 对导弹 M 的遮蔽过程是一个连续的过程，遮蔽过程的开始和结束各有两种情况：

- 开始时，可能烟幕云团 $D_{i,j}$ 恰与线段 MA 相切，记此情况为“切入”；也有可能烟幕云团 $D_{i,j}$ 与线段 MA 相交，此情况中烟幕弹突然爆炸形成的烟幕云团中心到线段 MA 的距离小于 10，记此情况为“突入”。
- 结束时，可能烟幕云团 $D_{i,j}$ 恰与线段 MA 相切，记此情况为“切出”；也有可能烟幕云团 $D_{i,j}$ 不与线段 MA 相切，但 $D_{i,j}$ 与 M 的距离大于 10，此种情况中，导弹突破烟幕云团，使得烟幕云团落后于导弹，无法继续形成有效遮蔽，记此情况为“突出”。

由于烟幕云团仅有恒定的竖直向下的速度，且导弹的速度大小方向固定，若以烟幕云团中心 $D_{i,j}$ 为参考系原点建立参考系 $x'D_{i,j}y'$ ，则可知在该参考系下导弹 D 仍在一条直线上匀速飞行。则每枚烟幕弹 $D_{i,j}$ 对导弹 M 的有效干扰可以分为四种情况，如图4所示：

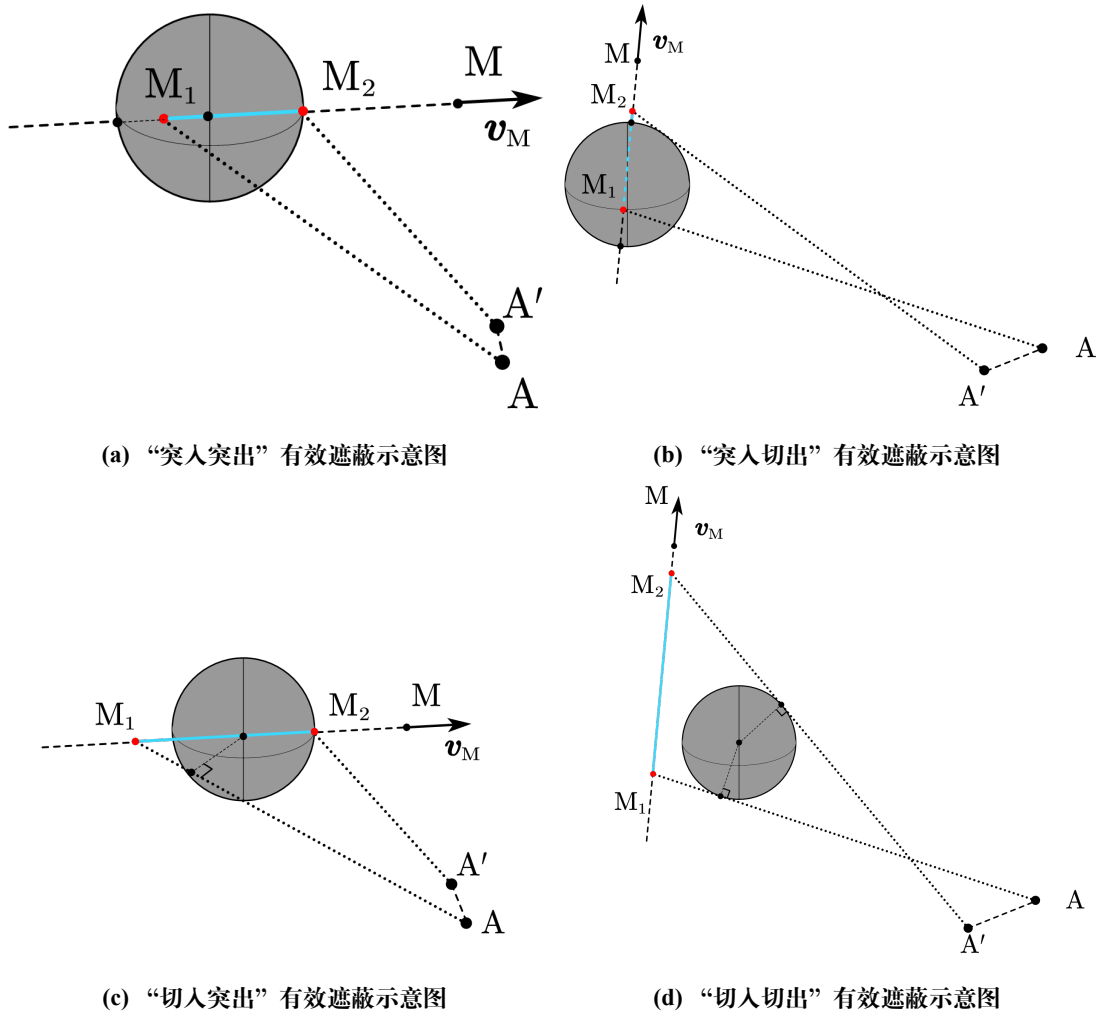


图4 四种有效遮蔽示意图

每个子图中，当 M 位于线段 M_1M_2 上时，烟幕云团形成有效遮蔽。在以烟幕云团中心 $D_{i,j}$ 为原点建立的坐标系中，真目标 A 具有竖直向上的恒定速度，在导弹 M 从 M_1 移动到 M_2 时，烟幕弹从 A 移动到 A' 。

对于这四种情况，“突入突出”情况可由“切入突出”情况调整烟幕弹的起爆位置得到，且显然后者的有效遮蔽时长更长；“突入切出”情况也可由“切入切出”情况调整起爆位置得到，同样后者的有效遮蔽时间大于前者；对于“切入突出”情况，总可以调整起爆位置，使得烟幕云团靠近 A 点，从而得到“切入切出”情况，并将有效遮蔽时间延长，如图 5 所示。

而“切入切出”的在本题目中的理想状态为两条切线平行。从一步地，可以由“切入切出”状态的理想情况近似求出每枚烟幕弹 $D_{i,j}$ 对任意导弹 M 的理论最大有效遮蔽时长。令 R 表示烟幕云团有效半径， $v_{fall} = 3 \text{ m/s}$ 表示烟幕云团下落的速度大小， γ_M 表示导弹 M 的高度角，则任意烟幕弹对导弹 M 的理论最大有效遮蔽时长为

$$t_{D_{i,j},M,max} = \frac{2R}{v_{fall} \cos \gamma_M} = \frac{20}{3 \cos \gamma_M} \quad (8)$$

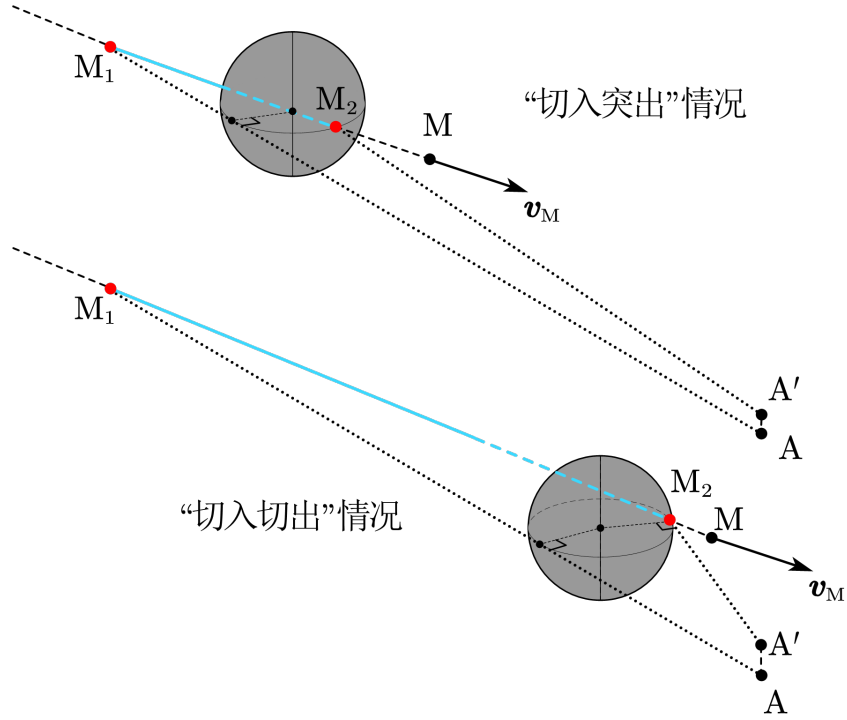


图5 “切入突出”情况转化为“切入切出”情况示意图

其中 γ_M 仅由导弹 M 的初始位置决定。

利用三枚导弹的初始位置求得任意烟幕弹对 M1、M2 和 M3 的理论最大有效遮蔽时长分别为 6.700s、6.707s 和 6.704s。

6.2 单无人机—单烟幕弹—单导弹模型的建立

问题二仍然仅考虑无人机 FY1 投放一枚烟幕干扰弹 $D_{1,1}$ 对 M1 进行干扰，但是需要确定 FY1 的飞行速度、飞行方向、烟幕弹的投放时间和起爆时间以求得最大的有效遮蔽时间。

令无人机 FY i 的飞行速度大小 v_{FYi} 、飞行方向角 α_{FYi} (以 x 轴为正向，逆时针方向为正，取值 $0 \sim 2\pi$)、飞行方向的单位向量 $\mathbf{s}_{FYi} = -[\cos \alpha_{FYi}, \sin \alpha_{FYi}, 0]^T$ 、烟幕弹的投放时间 $t_{D_{i,j},thr}$ 、起爆时间 $t_{D_{i,j},bst}$ ，给定上述变量的取值时，可以联立式(1)、(2)、(3)、(4)、(5)和式(6)解得烟幕弹 $D_{i,j}$ 对 M 的有效遮挡时间 $t_{D_{i,j},M}$ 。将此给值求解的过程定义为函数 f ，则有：

$$t_{D_{i,j},M} = f(v_{FYi}, \alpha_{FYi}, t_{D_{i,j},thr}, t_{D_{i,j},bst}, \mathbf{P}_{FYi,0}, \mathbf{P}_{M,0}) \quad (9)$$

则问题二的目标函数可以表示为：

$$t_{D_{1,1},M1} = f(v_{FY1}, \alpha_{FY1}, t_{D_{1,1},thr}, t_{D_{1,1},bst}, \mathbf{P}_{FY1,0}, \mathbf{P}_{M1,0})$$

其中 $\mathbf{P}_{FYi,0}$ 表示无人机 FY i 的初始位置， $\mathbf{P}_{M,0}$ 表示任意导弹 M 的初始位置。则可将问

题二转化为优化模型：

$$\begin{aligned} \max \quad & f(v_{FY1}, \alpha_{FY1}, t_{D_{1,1},thr}, t_{D_{1,1},bst}, \mathbf{P}_{FY1,0}, \mathbf{P}_{M1,0}) \\ \text{s.t.} \quad & \begin{cases} 70 \leq v_{FY1} \leq 140 \\ 0 \leq \alpha_{FY1} \leq 360^\circ \\ t_{D_{1,1},thr} \geq 0, t_{D_{1,1},bst} \geq t_{D_{1,1},thr} \end{cases} \end{aligned} \quad (10)$$

6.3 单无人机—单烟幕弹—单导弹模型的求解

6.3.1 减少需要优化的参数

导弹 M1 的位置矢量为 $\mathbf{r}_{M1}(t) = \mathbf{P}_{M1,0} + 300t \cdot \mathbf{s}_{M1} \quad t \geq 0$ ，由于 $\mathbf{P}_{M1,0}$ 与 $\mathbf{s}_{M1}$ 在 y 方向的分量为 0，则 M1 始终位于 xOz 平面。同理可知烟幕弹 $D_{1,1}$ 亦始终位于 xOz 平面。则由本问题模型准备知，要使得烟幕 $D_{1,1}$ 对 M1 的有效遮蔽时间最长，需要其有效遮蔽情况尽可能为“切入切出”。

再在参考系 $x'D_{1,1}y'$ 中对 M1 对 $D_{1,1}$ 的有效遮蔽进行分析，如图 6 所示。当导弹 M1 的运动方向为靠近 A 时，可以通过改变烟幕云团中心的位置远离 A 点使得其有效遮蔽时间延长。

继续考虑无人机 FY1 的速度大小 v_{FY1} 和投放时间 $t_{D_{1,1},thr}$ 对烟幕云团中心到 A 点距离的影响。如图 7 和图 8 所示。

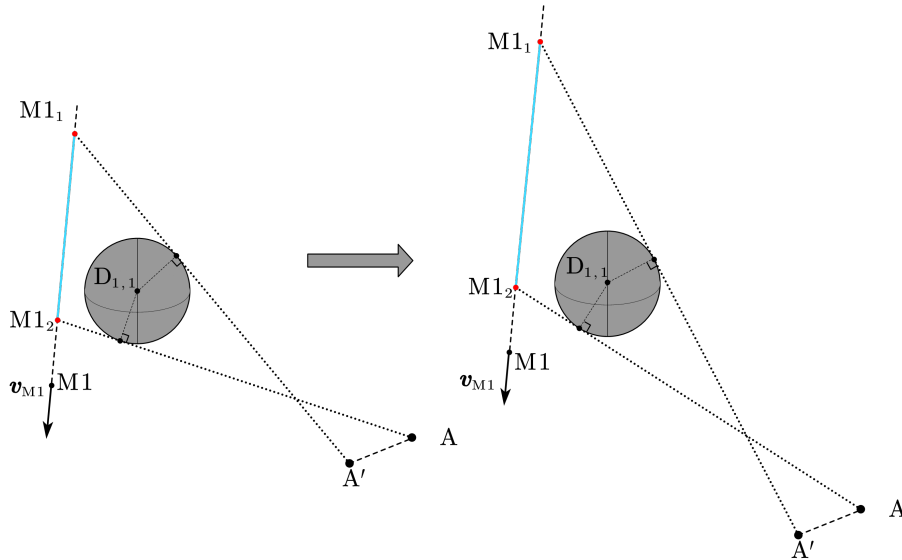


图 6 问题二最优示意图

如右图所示，轨迹 1 和轨迹 2 是无人机 FY1 以不同速度飞行投出的烟幕弹的运动轨迹，轨迹 2 对应的飞行速度大于轨迹 1 对应的飞行速度，因此其爆炸产生的烟幕云团更接近 A 点。

为了使得遮挡时间尽可能长，则需要烟幕弹爆炸时尽可能远离 A 点，即需要无人机 FY1 的速度尽可能小。

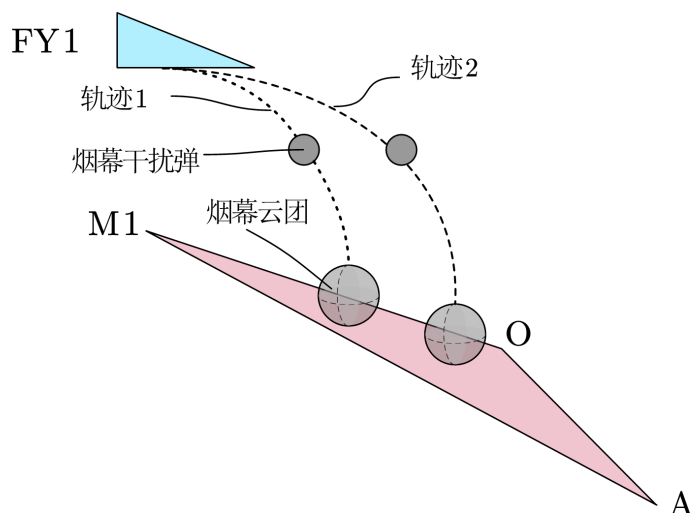
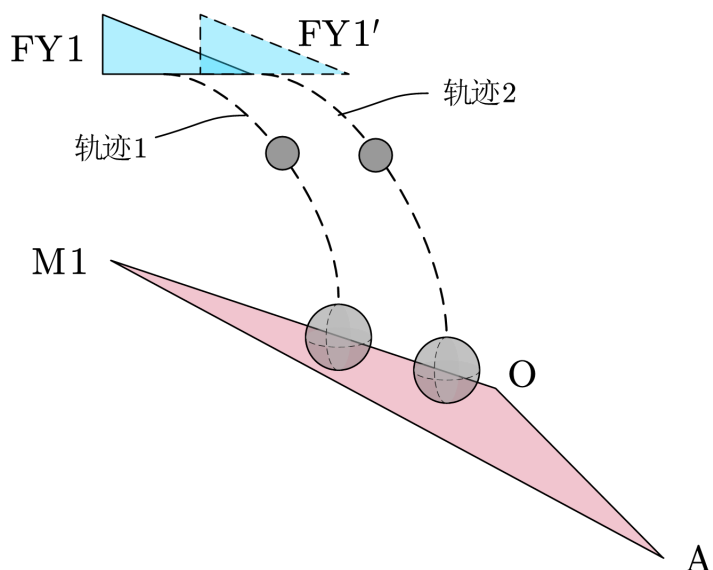


图 7 无人机速度大小对有效遮蔽时间的影响



如左图所示，FY1 以同样的速度飞行，并在不同的时刻投下烟幕弹，轨迹 2 的投弹时刻晚于轨迹 1，因此轨迹 2 对应的烟幕云团离 A 点更近。

为了使得遮挡时间尽可能长，则需要烟幕弹爆炸时尽可能远离 A 点，即需要无人机 FY1 投下烟幕干扰弹的时刻尽可能早。

图 8 投放时间对有效遮蔽时间的影响

可以得到，要使烟幕弹 $D_{1,1}$ 对导弹 M1 产生的有效遮蔽时间最大，需要使无人机 FY1 的速度值 v_{FY1} 和投下烟幕弹 $D_{1,1}$ 的时间 $t_{D_{1,1},bst}$ 最小。

为了给上述结论提供数据支撑，现对上述结论进行简单验证：固定起爆时间 $t_{D_{1,1},bst} = 3.6 \text{ s}$ 和飞行方向角 $\alpha_{FY1} = 176.6^\circ$ ，分别控制无人机飞行速度大小 v_{FY1} 和投放时间 $t_{D_{1,1},thr}$ 发生改变，对该过程进行灵敏度分析，研究有效遮蔽时长的变化趋势，如图 9 所示。可以看到，有效遮蔽时长的变化趋势与之前的叙述保持一致。

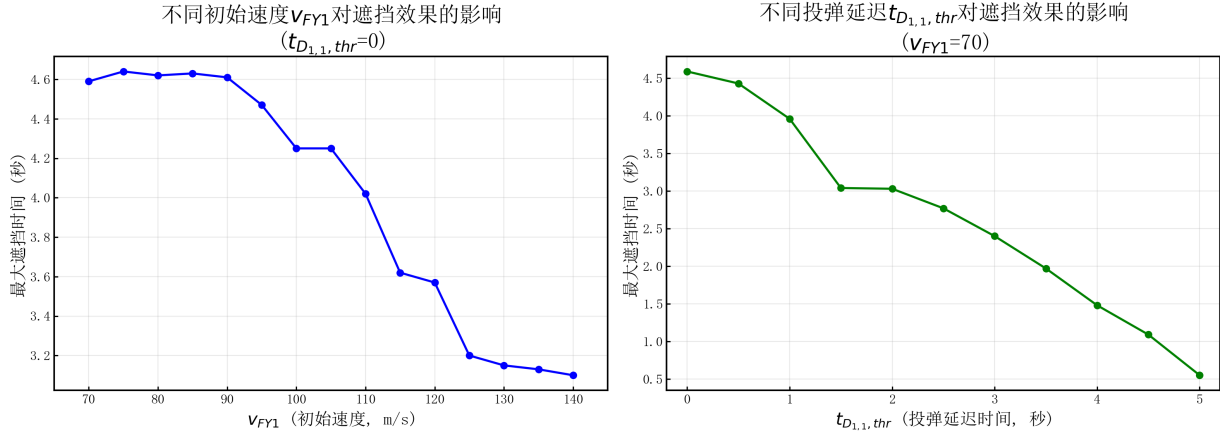


图 9 遮挡时间的变化

6.3.2 通过局部搜索得到其全局最优解

在减少了决策变量 v_{FY1} 和 $t_{D_{1,1},thr}$ 后, 需要针对无人机 FY1 方位角 α_{FY1} 和起爆时间 $t_{D_{1,1},bst}$ 进行优化。我们可以尝试在 MIOA 平面附近进行寻找, 可以通过以下的算法实现:

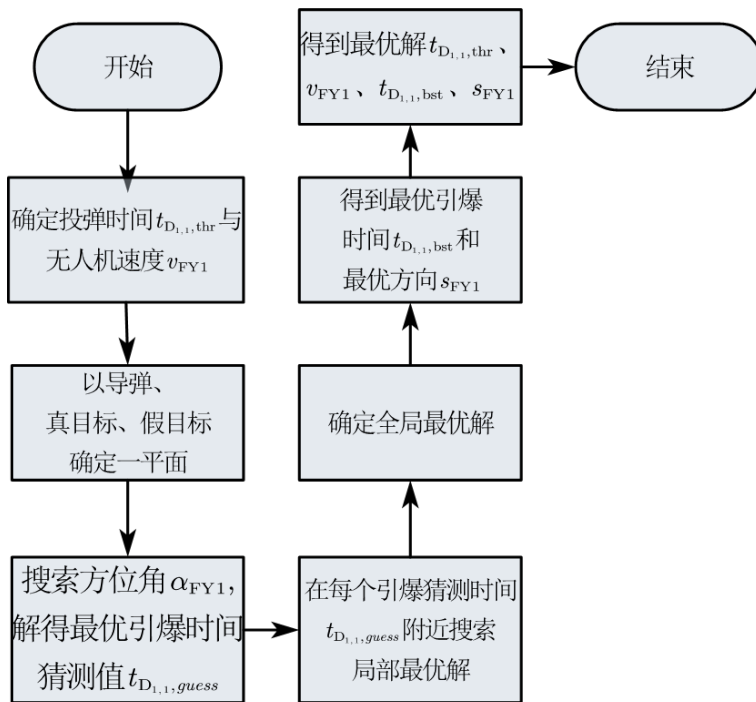


图 10 搜索算法流程图

STEP1: 由减少优化参数量时的分析, 取 $t_{D_{1,1},thr} = 0s$, $v_{FY1} = 70m/s$;

STEP2: 对每个方位角, 根据式 (7) 求得烟幕弹从开始投弹到其首次到达距离 MIOA 平面距离十米的时间, 该时间为最优起放时间的猜测值;

STEP3: 对每个方位角对应的最优起放时间猜测值进行局部搜索, 得到每个方位角下的局部最优解;

STEP4: 对比每个搜索到的局部最优解, 求得全局最优解。

6.4 求解结果

优化后的参数为：无人机FY1飞行速度 v_{FY1} m/s, 烟幕弹投放时间 $t_{D_{1,1},thr} = 0$ s, 无人机飞行的方向向量 $\mathbf{s} = [-0.9983, 0.0585, 0]^T$, 起爆间隔时间 $t_1 = 2.467$ s. 代入上述参数, 取时间步长 0.001 s 进行仿真模拟, 得到烟幕干扰弹的投弹坐标为 $[17800, 0, 1800]^T$, 烟幕干扰弹起爆点坐标为 $[17627.61, 10.11, 1770.18]^T$, 遮蔽时间段为 2.487 s $\sim$ 7.167 s, 因此 **最长有效遮蔽时间** 为: $t_{\max}=4.680$ s.

6.5 结果分析

得到的最长有效遮蔽时间为 4.680 s, 较为接近理论最大值 6.700 s, 说明求解效果良好。在该组参数下烟幕弹对导弹的遮蔽情况如图 11所示, 遮蔽开始时刻遮蔽情况如图 11a所示, 导弹与真目标连线的线段和烟幕云团的边缘恰好相切, 此时刚好遮蔽; 如图 11b所示, 导弹与真目标连线的线段和烟幕云团的边缘恰好相切, 此时恰好不遮蔽。

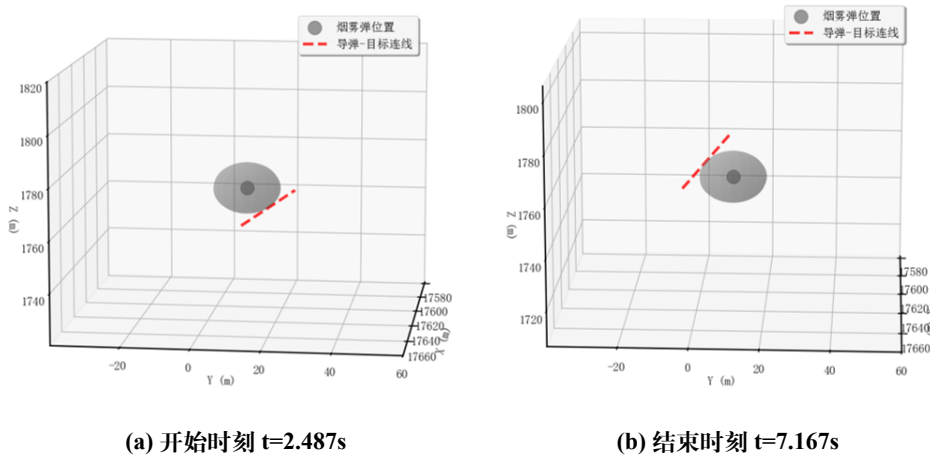


图 11 遮蔽始末烟幕弹遮蔽情况

七、问题三的模型的建立和求解

7.1 模型准备

要求解本题, 需要定义“单无人机—多烟幕弹—单导弹策略的总遮蔽时长”。对于每个“单无人机—多烟幕弹—单导弹策略”, 可以将其拆分为 3 个(单无人机携带投弹数目个)“单无人机—单烟幕弹—单导弹”策略。即:

定义 2 (单无人机—多烟幕弹—单导弹策略总有效遮蔽时长). 本文定义为所有“单无人机—单烟幕弹—单导弹”策略有效遮蔽时长之和

在有多枚烟幕弹参与干扰时, 要使得多枚烟幕弹对真目标的有效遮蔽时间尽可能长, 可以 **让每枚烟幕弹的有效遮蔽时间尽可能长**, 由于同一架无人机 FY_i 上的烟幕弹

共享无人机运动参数 v_{FYi} 与 α_{FYi} ，且投放时间需要间隔 1s，所以可以将目标转化为使**多枚烟幕弹对真目标的有效遮蔽时间之和尽可能长**。

在计算上，若将每枚烟幕弹对导弹的有效遮蔽时间分别表示为一个区间，则需要使各烟幕弹有效遮蔽区间的测度之和最大。

7.2 单无人机多烟幕弹单导弹模型的建立

在考虑第一枚烟幕弹的投放时，可以直接利用**问题二模型最优解**中的投放时间和起爆时间作为第一枚烟幕弹的投放时间 $t_{D_{1,1},thr}$ 和起爆时间 $t_{D_{1,1},bst}$ ，同时确定无人机 FY1 的飞行速度 v_{FY1} 和飞行方位角 α_{FY1} 。

改写函数 (9)，定义向量函数 $\mathbf{g} : \mathbb{R}^{10} \rightarrow \mathbb{R}^2$ ，用以表示给定参数时求解开始遮挡的时间与结束遮挡的时间的过程：

$$[t_{i,j,start}, t_{i,j,end}] = \mathbf{g}(v_{FYi}, \alpha_{FYi}, t_{D_{i,j},thr}, t_{D_{i,j},bst}, \mathbf{P}_{FYi,0}, \mathbf{P}_{M,0}) \quad (11)$$

定义映射 $\mathcal{T} : \mathbb{R}^2 \rightarrow \mathcal{P}(\mathbb{R})$ ，将向量转化为可测点集，便于表示区间长度：

$$\mathcal{T}(x, y) = \{t | x \leq t \leq y\}$$

以 $m(\cdot)$ 表示可测点集的测度，则三枚烟幕弹对 M1 的有效遮蔽时长之和可以表示为：

$$\sum_{i=1}^3 m \circ \mathcal{T} \circ \mathbf{g}(v_{FY1}, \alpha_{FY1}, t_{D_{1,i},thr}, t_{D_{1,i},bst}, \mathbf{P}_{FY1,0}, \mathbf{P}_{M1,0})$$

则可以将问题三转化为优化问题：

$$\begin{aligned} \max \quad & \sum_{i=1}^3 m \circ \mathcal{T} \circ \mathbf{g}(v_{FY1}, \alpha_{FY1}, t_{D_{1,i},thr}, t_{D_{1,i},bst}, \mathbf{P}_{FY1,0}, \mathbf{P}_{M1,0}) \\ \text{s.t.} \quad & \begin{cases} 70 \leq v_{FY1} \leq 140 \\ 0 \leq \alpha_{FY1} < 360^\circ \\ t_{D_{1,i},thr} \geq 0 & i = 1, 2, 3 \\ t_{D_{1,i},bst} \geq 0 & i = 1, 2, 3 \\ t_{D_{1,i},thr} \geq 1 + t_{D_{1,i-1},thr} & i = 2, 3 \end{cases} \end{aligned} \quad (12)$$

7.3 单无人机—单烟幕弹—单导弹模型的求解

需要考虑的变量的有无人机飞行速度 v_{FY1} 、飞行方向角 α_{FY1} 以及三枚烟幕弹的投放时间 $t_{D_{1,i},thr}$ ($i = 1, 2, 3$) 和起爆时间 $t_{D_{1,i},bst}$ ($i = 1, 2, 3$)，总共八个变量。

为了使每枚烟幕弹的有效遮蔽时间最长，同时减少需要优化的参数量，可以**利用贪心算法减少对参数的计算**：对于每一枚烟幕弹，都仅考虑如何使该烟幕弹在投放起爆后

的总有效遮蔽时间最长，无需考虑后续烟幕弹的投放起爆情况。又由问题二模型准备中的分析得，每枚烟幕弹的投放时间越小越好，则可尝试令每枚烟幕弹的投放时间恰好间隔 1s。

可以利用贪心算法求解模型，具体算法如下：

Step1: 利用问题二中的求解结果确定第一枚烟幕弹的投放时间 $t_{D_{1,1},thr}$ 和起爆时间 $t_{D_{1,1},bst}$ ，同时确定无人机 FY1 的飞行速度 v_{FY1} 和飞行方位角 α_{FY1} ；

Step2: 确定第二个烟幕弹的投放时间猜测值为 $t_{D_{1,2},thr} = t_{D_{1,1},thr} + 1$ ，利用局部网格搜索确定第二个烟幕弹的起爆时间 $t_{D_{1,2},bst}$ ，使得第二个烟幕弹的遮蔽区间的测度最大；

Step3: 同 **Step2**，确定第三个烟幕弹的投放时间猜测值为 $t_{D_{1,3},thr} = t_{D_{1,2},thr} + 1$ ，利用局部网格搜索确定第三个烟幕弹的起爆时间 $t_{D_{1,3},bst}$ ，使得第三个烟幕弹的遮蔽区间测度最大，然后求得有效遮蔽时长之和的最大值。

7.4 求解结果

利用贪心算法求解得到的投放策略及结果如表 1 所示，求得最长总有效遮蔽时长为 7.56 秒。

表 1 投放三枚烟幕干扰弹相关参数表

无人机运动方向	无人机速度 (m/s)	烟幕弹编号	烟幕弹投放坐标 (m)	烟幕弹起爆坐标 (m)	有效干扰时长 (s)
176.64°	70	1	(17800, 0, 1800)	(17628, 10, 1770)	4.68
176.64°	70	2	(17730, 4, 1800)	(17542, 15, 1764)	2.88
176.64°	70	3	(17660, 8, 1800)	(17660, 8, 1800)	0

八、问题四的模型的建立和求解

8.1 模型准备

要求解本题，需要定义“多无人机—单烟幕弹—单导弹”策略的总遮蔽时长。对于一个“多无人机—单烟幕弹—单导弹”策略，可以将其拆分为多个(无人机的数目个)“单无人机—单烟幕弹—单导弹”策略。即：

定义 3 (多无人机—单烟幕弹—单导弹策略的总遮蔽时长)，本文定义为所有“单无人机—单烟幕弹—单导弹”策略的总遮蔽时长之和

8.2 多无人机单烟幕弹单导弹模型的建立

问题四要求利用 FY1、FY2 和 FY3 三架无人机各投一枚烟幕弹对 M1 进行干扰，为多无人机单烟幕弹单导弹模型。由于三架无人机进行一次投弹的策略各自完全独立，则

可以在问题二模型的基础上进行分析, 将本问题拆分为三个“单无人机—单烟幕弹—单导弹”问题, 并求各自有效遮蔽时长的最大值, 以减少计算量。

由式(12), 对于“单无人机—单烟幕弹—单导弹”问题, 第 i 架无人机对导弹 M1 的有效遮蔽时长为:

$$m \circ \mathcal{T} \circ \mathbf{g}(v_{\text{FY},i}, \alpha_{\text{FY},i}, t_{D_{i,1},\text{thr}}, t_{D_{i,1},\text{bst}}, \mathbf{P}_{\text{FY},i,0}, \mathbf{P}_{\text{M1},0})$$

为了使得总遮蔽时长最大, 根据定义3, 可建立优化模型:

$$\begin{aligned} \max \quad & \sum_{i=1}^3 m \circ \mathcal{T} \circ \mathbf{g}(v_{\text{FY},i}, \alpha_{\text{FY},i}, t_{D_{i,1},\text{thr}}, t_{D_{i,1},\text{bst}}, \mathbf{P}_{\text{FY},i,0}, \mathbf{P}_{\text{M1},0}) \\ \text{s.t.} \quad & \begin{cases} 70 \leq v_{\text{FY},i} \leq 140 & i = 1, 2, 3 \\ 0 \leq \alpha_{\text{FY},i} < 360^\circ & i = 1, 2, 3 \\ t_{D_{i,1},\text{thr}} \geq 0, t_{D_{i,1},\text{bst}} \geq 0 & i = 1, 2, 3 \end{cases} \end{aligned} \quad (13)$$

上述优化模型需要优化参数数目达到 12 个, 求解难度非常大, 若直接使用各类优化算法极易得到局部最优解, 解的质量难以保证。为了减少优化参数数目, 给出以下命题及简要证明。

命题 1. 基于定义3, 当每个无人机在“单无人机—单烟幕弹—单导弹”投弹策略下的遮蔽时长都取到最大值时, 总的有效遮蔽时间(多无人机—单烟幕弹—单导弹投弹策略的总遮蔽时长)取到最大。

证明: 根据定义3分析可知, 是由多个“单无人机—单烟幕弹—单导弹”投弹策略的遮蔽时长之和。而根据问题 2 中的优化模型(10), 对于任意一个无人机在该策略下的最优遮蔽时长不受其他无人机的运动参数与投弹参数影响, 即可证明命题 1。

□

由命题1, 可以将(13)拆解为 3 个单目标优化问题:

$$\begin{aligned} \max \quad & m \circ \mathcal{T} \circ \mathbf{g}(v_{\text{FY},i}, \alpha_{\text{FY},i}, t_{D_{i,1},\text{thr}}, t_{D_{i,1},\text{bst}}, \mathbf{P}_{\text{FY},i,0}, \mathbf{P}_{\text{M1},0}) \quad i = 1, 2, 3 \\ \text{s.t.} \quad & \begin{cases} 70 \leq v_{\text{FY},i} \leq 140 & i = 1, 2, 3 \\ 0 \leq \alpha_{\text{FY},i} < 360^\circ & i = 1, 2, 3 \\ t_{D_{i,1},\text{thr}} \geq 0, t_{D_{i,1},\text{bst}} \geq 0 & i = 1, 2, 3 \end{cases} \end{aligned}$$

每个优化问题仅需优化求解 4 个优化参数, 并且可以利用问题 2 中的算法, 对这 3 个优化问题进行求解, 能够大幅减少计算成本, 提高解的质量。

8.3 模型求解

对三架无人机各 4 个参数进行优化, 同样有较大的计算量, 继续考虑减少需要优化的参数数量。对于 FY1, 可以使用问题二模型的解作为其最优解, 则仅需对 FY2 与 FY3

进行优化。

进一步地，为了缩小优化参数的范围，可以分析 $FY_i(i=2,3)$ 飞行方位角对有效遮蔽时长的影响。

对于无人机的飞行方向角，由问题二模型求解中减少优化参数的分析可知，每架无人机投下的烟幕弹形成有效遮蔽时，烟幕云团的中心距离真目标 A 越远越好，所以需要考虑让无人飞行的方向远离 A ；此外，由于每架无人机的飞行速度大幅度低于导弹的飞行速度，需要考虑让无人机尽可能快地到达导弹飞行轨迹附近。综合考虑二者，可选取垂直于线段 $P_{M,0}A$ 且靠近导弹 M 的方向对应的方向角作为无人机的飞行方位角猜测值。

确定各无人机飞行方位角的粗略范围后，可以用以下算法对每架无人机的 4 个参数进行求解：

Step1: 采用遗传算法，将飞行方位角约束在猜测值附近进行粗略搜索，如图12所示。

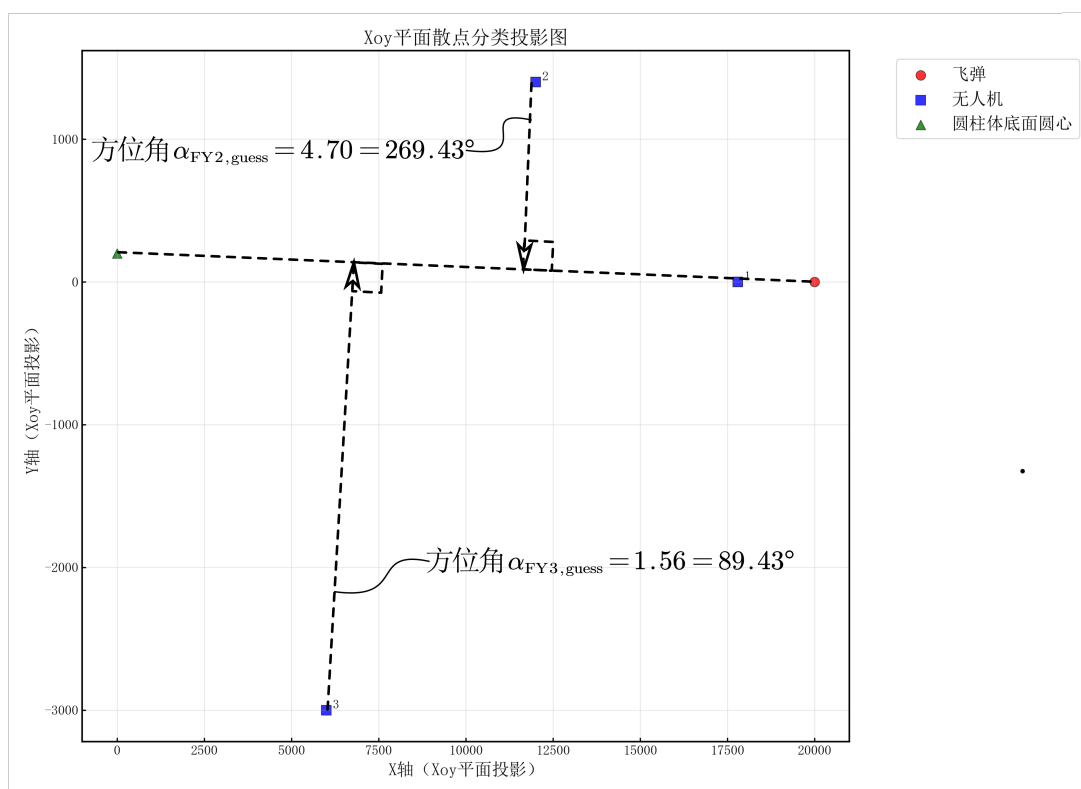


图 12 遗传算法搜索无人机参数示意图

Step2: 对于遗传算法得到的可行解，进行网格局部高精度搜索最优飞行参数与导弹参数。

8.4 求解结果

求解得各无人机投放的烟幕弹的有效遮蔽时间区间如表 2所示，各无人机投放烟幕弹的投放点与起爆点的坐标如表 3所示，FY1 投放的烟幕弹遮蔽 M1 始末情况如图 11所示，FY2 和 FY3 投放的烟幕弹遮蔽 M1 始末情况如图 13所示，各烟幕弹有效遮蔽时长之和最大值为 13.91 秒。

表 2 三架无人机烟幕干扰弹遮蔽时间区间

无人机编号	无人机运动方向	无人机运动速度 (m/s)	遮蔽起始时间 (s)	遮蔽结束时间 (s)	有效干扰时长 (s)
FY1	176.6°	70	2.49	7.17	4.68
FY2	272.2°	124	15.02	19.54	4.52
FY3	101.9°	105.44	30.14	34.85	4.71

表 3 三架无人机烟幕干扰弹相关参数表

无人机编号	无人机运动方向	无人机运动速度 (m/s)	烟幕干扰弹投放点的坐标 (m)	烟幕干扰弹起爆点的坐标 (m)	有效干扰时长 (s)
FY1	176.6°	70	(17800, 0, 1800)	(17627, 10, 1770)	4.68
FY2	272.2°	124	(12023, 780, 1400)	(12051, 37, 1223)	4.52
FY3	101.9°	105.44	(5473, -489, 700)	(5350, 96, 542)	4.71

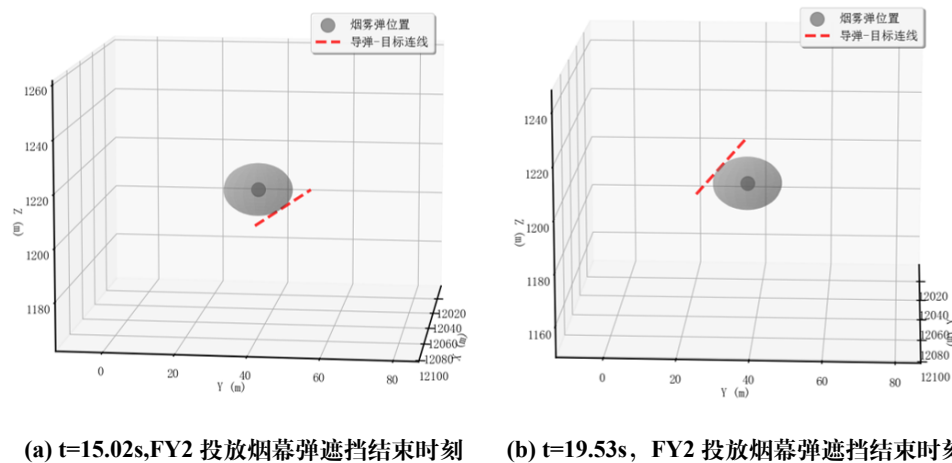
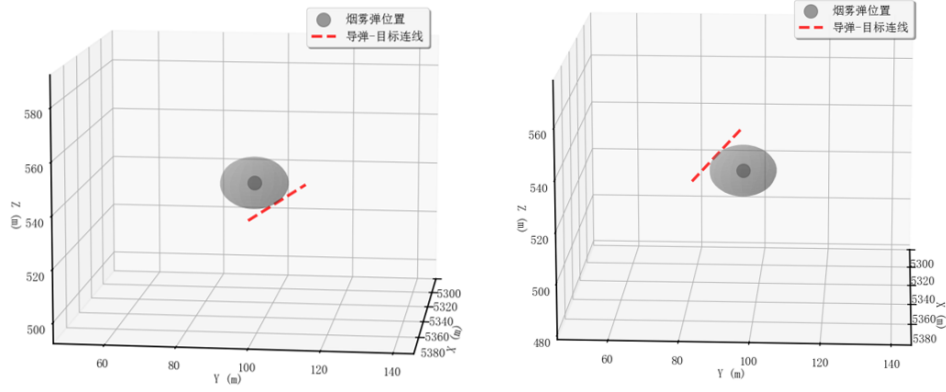


图 13 FY2 与 FY3 投弹遮蔽始末情况



(c) $t=30.13s$, FY3 投放烟幕弹遮挡开始时刻 (d) $t=34.84s$, FY3 投放烟幕弹遮挡结束时刻

图 13 FY2 与 FY3 投弹遮蔽始末情况 (续)

8.5 结果分析

各无人机投放烟幕弹的有效干扰时长与问题二相近,说明模型的性能良好。将求解结果可视化并进行分析,如图14所示,由此提出的猜想可以为后续优化问题提供参考,缩小对于无人机方位角的搜索范围,大幅提升优化的速率与成功率。

如右图所示,求解得到的方位角近乎垂直于 M1 与真目标底部的连线,与本问题模型求解处的分析吻合得很好。基于此,可以给出以下猜想:对于距离导弹 M 较远的无人机,其飞行方位角的最优解对应的飞行方向垂直于导弹 M 的初始位置与真目标 A 的连线 $P_{M,0}$ 。

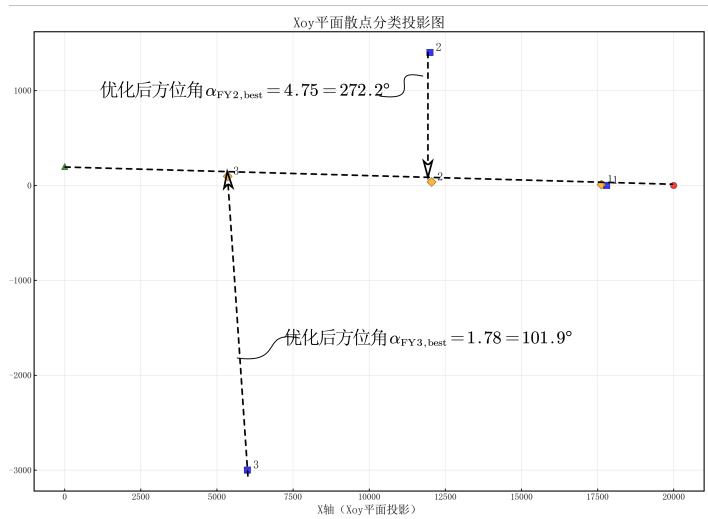


图 14 求解得到的 FY2,FY3 最优方位角

九、问题五的模型的建立和求解

9.1 模型准备

要求解本题,需要定义“多无人机—多烟幕弹—多导弹策略的总遮蔽时长”。若考虑每个飞机均投出 3 枚烟幕弹,每枚烟幕弹仅干扰 1 颗导弹,则对于一个“多无人机—多

烟幕弹—多导弹策略”，可以将其拆分为 3 个 (投放烟幕弹/导弹数目) 多无人机—单烟幕弹—单导弹策略。即：

定义 4(多无人机—多烟幕弹—多导弹策略的总遮蔽时长). 本文定义为所有多无人机—单烟幕弹—单导弹策略的总遮蔽时长之和

9.2 模型的建立

问题五要求五架无人机各投放至多三枚烟幕弹对 M1、M2 和 M3 进行干扰，共涉及到 40 个参数的优化。由于各架无人机的投弹策略是独立的，为了减少需要优化的参数量，我们可以尝试对每个无人机的投弹策略进行独立分析。

对于**单无人机的投弹问题**，可以根据要遮蔽的导弹数目，将其拆解为**单无人机多烟幕弹单导弹问题** (可利用问题三的模式求解) 或**单无人机多烟幕弹多导弹问题**。又由于各导弹之间的距离远大于烟幕云团的直径，知烟幕云团几乎不可能同时出现在两条导弹的飞行轨迹上，则可以将**单无人机多烟幕弹多导弹问题拆解为有时间约束的“单无人机—单烟幕弹—单导弹”问题** (可利用问题二的模式求解)。

因此，根据定义 4，可将问题五转化为以下优化模型：

$$\begin{aligned}
 \max \quad & \sum_{j=1}^3 \sum_{i=1}^5 \sum_{k=1}^3 m \circ \mathcal{T} \circ g(v_{FY,i}, \alpha_{FY,i}, t_{D_{i,j},thr}, t_{D_{i,j},bst}, P_{FY,i,0}, P_{Mk,0}) \\
 \text{s.t.} \quad & \begin{cases} 70 \leq v_{FY,i} \leq 140 & i = 1, 2, 3 \\ 0 \leq \alpha_{FY,i} \leq 2\pi & i = 1, 2, 3 \\ t_{D_{i,j},thr} \geq 0, t_{D_{i,j},bst} \geq 0 & i, j = 1, 2, 3 \\ t_{D_{i,j},thr} \geq 1 + t_{D_{i,j-1},thr} & i = 1, 2, 3; j = 2, 3 \end{cases} \quad (14)
 \end{aligned}$$

9.3 模型的求解

求解模型时，为了减小计算量，仍然可以采用贪心的思想，并对前面各问题的解进行复用，逐步求出每架无人机的投弹策略。具体算法如下：

STEP1: 制定拟投弹策略并优化 FY1

由于无人机 FY1 与导弹 M1 始终位于同一平面上，令无人机 FY1 仅对 M1 进行干扰，将 FY1 的投弹问题处理为单无人机多烟幕弹单导弹问题，其投弹策略按照问题三求解得到的策略进行；对于其余无人机，根据问题四中求解结果的分析 (如图 12)，令其余每架无人机的飞行方向近似垂直于各导弹初始位置与真目标 A 的连线段；每架无人机投出 3 枚烟幕弹，分别尝试遮挡导弹 M1，M2，M3。得到拟投弹策略如图 15 所示。

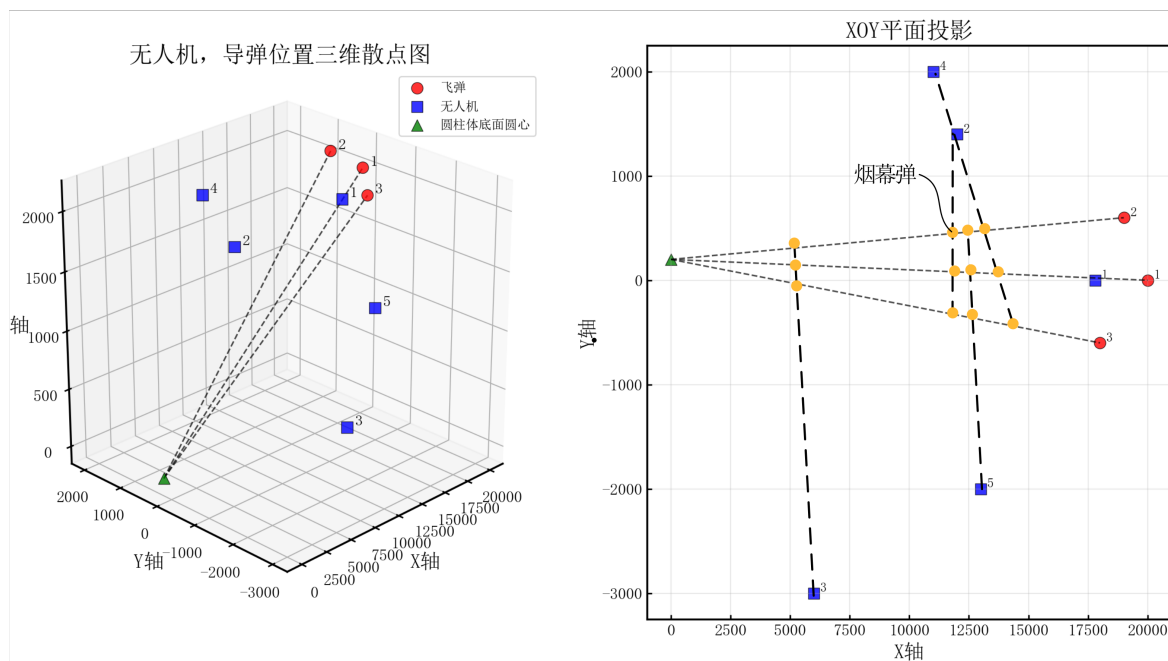


图 15 拟投弹策略示意图^[1]

STEP2: 优化 FY2,FY3 的投弹参数

分别将 FY2、FY3 的投弹问题处理为单无人机—多烟幕弹—多导弹问题：在 **STEP1** 的基础上，可以令无人机 FY2,FY3 的飞行参数和某一枚烟幕的投弹参数与问题四求解结果一致，然后利用有投弹时间约束的问题二模型优化求解 FY2 和 FY3 的另外两外两枚烟幕弹的投弹参数。

STEP3: 优化 FY4,FY5 的运动参数和投弹参数

分别将 FY4、FY5 的投弹问题处理为单无人机多烟幕弹多导弹问题：首先根据问题四结果分析中的猜想缩小方位角的搜索范围；再根据问题二模型的求解算法，确定无人机 FY4,FY5 的飞行参数和某一枚烟幕的投弹参数；最后同 **STEP2**，利用有投弹时间约束的问题二模型，优化求解 FY4 和 FY5 的另外两外两枚烟幕弹的投弹参数。

9.4 求解结果

表 4 问题 5 求解结果

无人机 编号	运动方向 (°)	运动速度 (m/s)	烟幕弹编 号	...	有效干扰时 长 (s)	干扰的导 弹编号
FY1	176.64	70	1	...	4.68	M1
			2	...	2.88	M1
			3	...	0.00	M1

续下页

表 4 问题 5 结果 (续)

无人机 编号	运动方向 (°)	运动速度 (m/s)	烟幕弹编 号	...	有效干扰时 长 (s)	干扰的导 弹编号
FY2	272.20	124	1	...	4.85	M2
			2	...	4.52	M1
			3	...	3.90	M3
FY3	101.90	105.44	1	...	4.03	M3
			2	...	4.71	M1
			3	...	4.03	M2
FY4	295.87	138.18	1	...	4.21	M2
			2	...	0.00	M1
			3	...	0.00	M3
FY5	92.69	140	1	...	0.00	M3
			2	...	4.39	M1
			3	...	0.00	M2

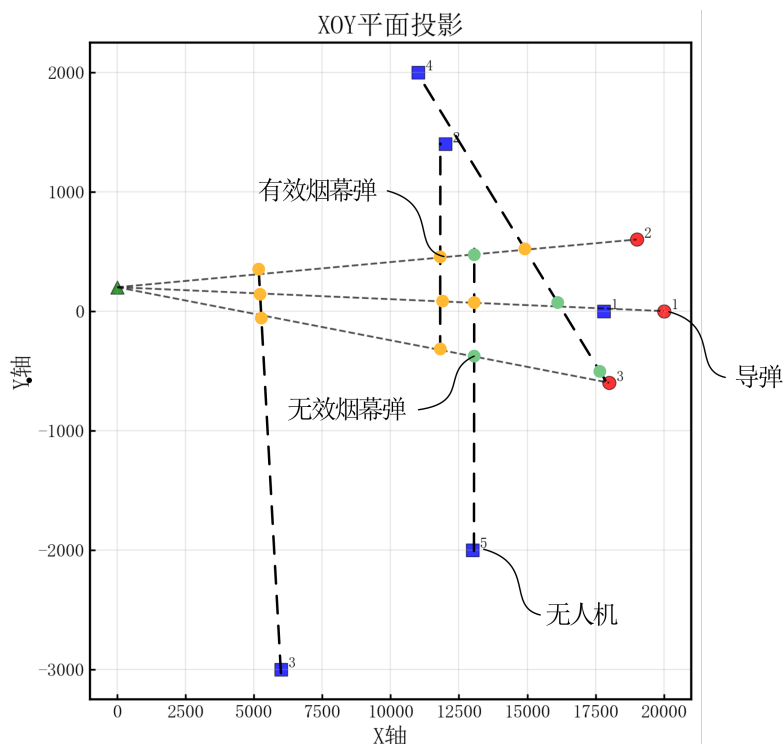
注：1. 表中“...”代表省略的烟幕弹投放坐标与起爆坐标参数，由于篇幅限制，本文不做展示，详见附件 result3.xlsx；

2. 该多行多列长表格格式由豆包 [2] 优化

问题 5 的求解结果如表4所示，其中，FY1 的运动参数与投弹参数直接使用的问题是三的求解结果对 M1 进行 3 次干扰。FY2 和 FY3 的运动参数和对 M1 干扰的投弹参数直接使用问题四的求解结果，进一步解出 FY2 和 FY3 对 M2,M3 的平均干扰时长均超过了 4.25s，遮挡效果较好。对于 FY4 只能求解出对 M2 的有效遮挡时间为 4.21s，其余两枚烟幕弹无效。对于 FY5 能求解出对 M1 的有效遮挡时间为 4.39s，求其余两枚烟幕弹无效。合计总共投出 15 枚烟幕弹，其中 10 枚烟幕弹可以形成有效遮蔽，总遮蔽时长为 42.20s，有效烟幕弹的平均遮蔽时间为 4.22s，所有烟幕弹的平均有效遮蔽时长为 2.813s。进一步地，可以求出所有导弹同时被遮蔽的时间段长度为 4.27s。

9.5 结果分析

将问题五的投弹策略及遮蔽效果可视化，如图 15所示：



如左图所示，为五架无人机投弹策略的最优解，图中黄色散点表示能够形成有效遮蔽的烟幕弹，绿色散点表示不能形成有效遮蔽的烟幕弹。由于FY1的结果与问题三种结果相同，因此图中只绘制了FY2,FY3,FY4,FY5的投弹策略及遮蔽效果，可以看到，该最优解的效果与图15表示的拟投弹策略效果相近，验证了问题四结果分析中猜想的正确性。

图 16 问题五投弹策略求解结果

将各无人机投放烟幕弹产生的有效遮蔽时长可视化，如图17所示。

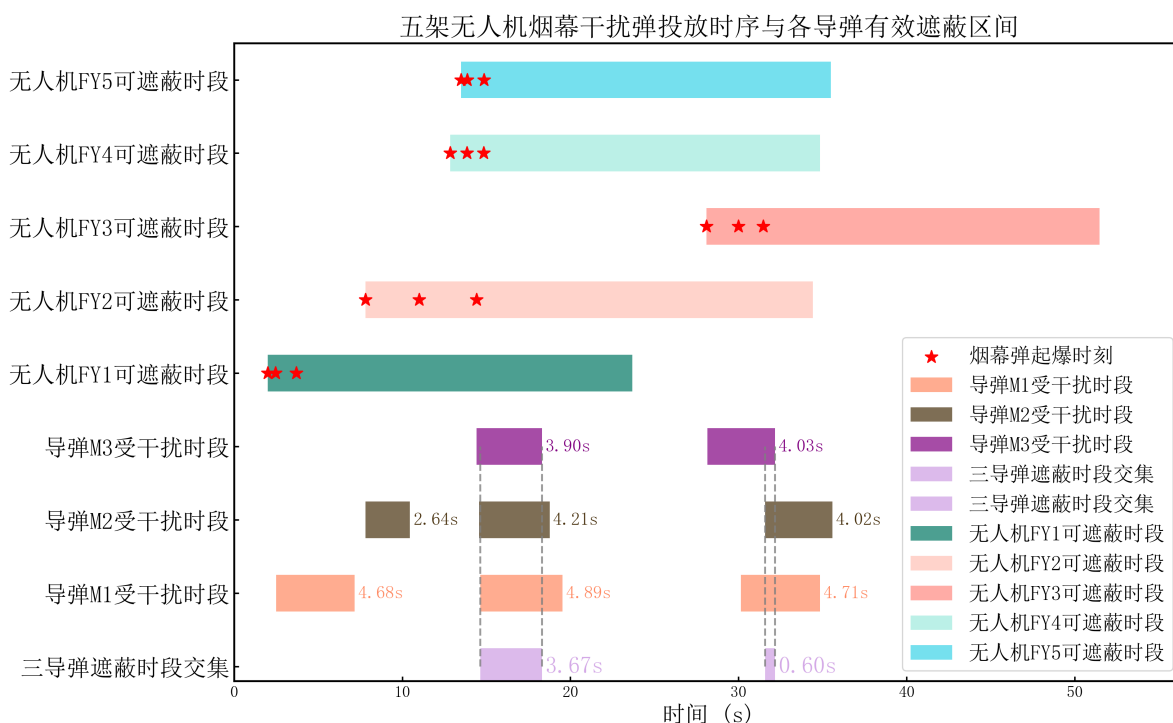


图 17 五架无人机对各导弹产生的有效遮蔽示意图^[2]

如图 17所示，每架无人机在红星对应的时刻引爆投下的烟幕弹，进入可能产生有

效遮蔽的时段，该时段在对应烟幕弹起爆 20s 后结束，可以重叠。对于每枚导弹，只有在可遮蔽时段中才可能被有效干扰，形成受干扰时段。当所有导弹的受干扰时段产生交集时，在此交集时段内，所有的导弹都被有效干扰。

十、模型的分析与检验

10.1 灵敏度分析

本文在求解各个问题的优化模型过程中，对参数的取值范围限定尤为重要。这里对无人机的飞行方位角 α_{FYi} 进行灵敏度分析，分别以无人机 FY2 干扰导弹 M1,FY3 干扰 M1,FY4 干扰 M2,FY5 干扰 M1 为例，除参数 α_{FYi} 外其余参数都取问题五中的求解结果，对于每个情形，以问题五中求解的 α_{FYi} 为中心，分别以 0.1, 0.2, 0.3, 0.5,0.8,1,1.2 为搜索半径探究有参数 α_{FYi} 对有效遮蔽时长的影响大小。如图 18所示。

可见，不管是在哪种情形下，**有效遮蔽时间（干扰时间）都对参数 α_{FYi} 极度敏感**：仅在极窄的方位角区间内，干扰时间才显著大于 0，形成“峰值带”，区间外干扰时间几乎为 0，说明方位角微小偏离“有效区间”时，干扰效果会急剧丧失。因此这就凸显出在问题二到问题五求解优化问题时根据几何分析确定飞行参数 α_{FYi} 的窄区间取值范围的重要性。

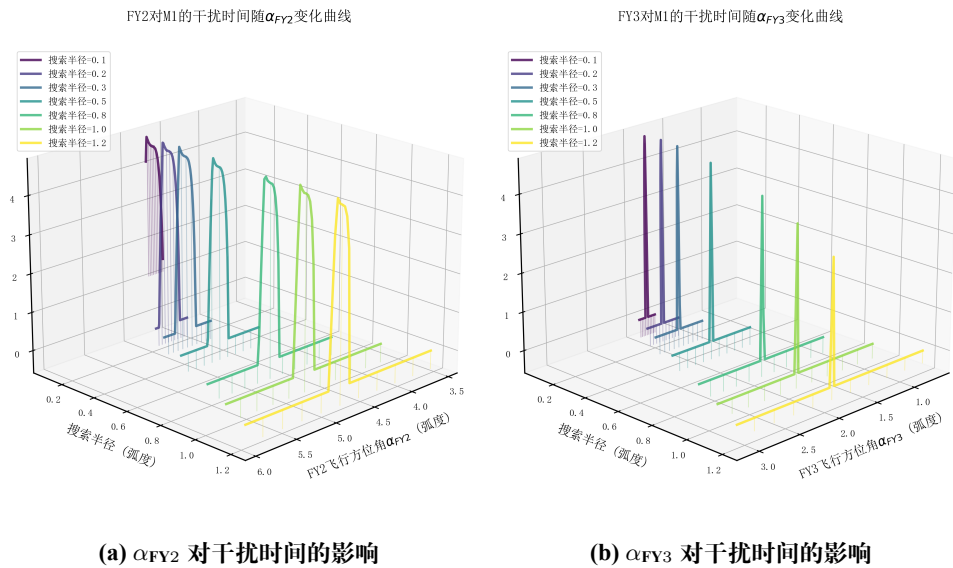


图 18 α_{FYi} 在不同情形在的灵敏度分析^[2]

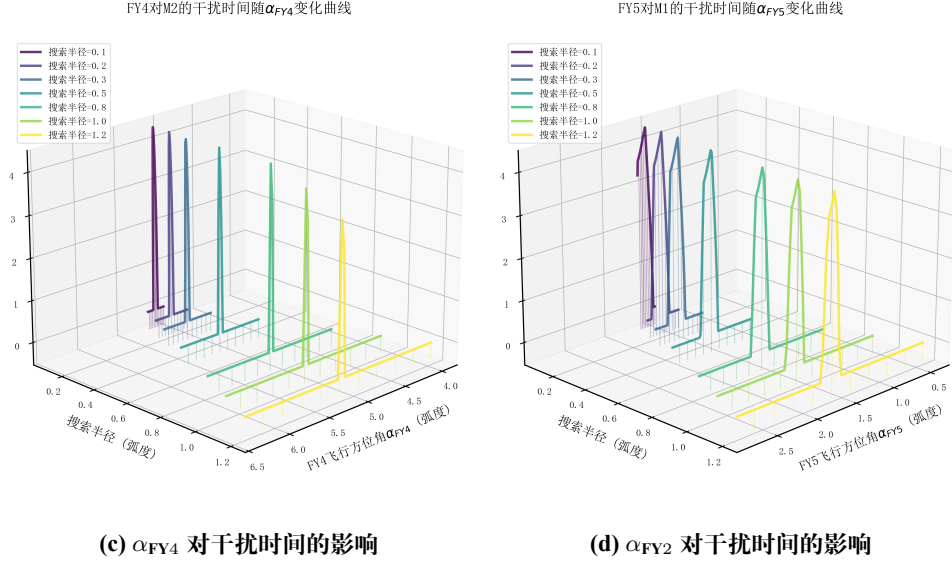


图 18 α_{FYi} 在不同情形下的灵敏度分析^[2] (续)

十一、模型的评价、改进

11.1 模型的优点

- 模型求解通过几何分析和智能算法确定参数大致范围，显著减小了参数搜索空间的大小，提高了参数搜索的效率。
- 模型求解通过几何分析与分布优化，减少了优化的变量数。
- 为了减少求解复杂度，将真目标视为质点。

11.2 模型的缺点

- 在该问题背景下，模型对参数过于敏感，且解存在的范围较为狭小，较难搜索到解。
- 将真目标视为质点，忽略了其真实体积。

11.3 模型的推广与改进

本文的模型建立是基于定义1,2,3,4建立的。若对于单无人机—多烟幕弹—单导弹问题和多无人机—单烟幕弹—单导弹问题，将总的有效遮蔽时长定义为各个烟幕弹遮挡时间段的并集的测度。则问题三中的优化模型(12)的目标函数（约束条件不变）应当修改为：

$$\max m \left[\bigcup_{i=1}^3 \mathcal{T} \circ g(v_{FY1}, \alpha_{FY1}, t_{D_{1,i},thr}, t_{D_{1,i},bst}, \mathbf{P}_{FY1,0}, \mathbf{P}_{M1,0}) \right]$$

若对于多无人机—多烟幕弹—多导弹问题，将总的有效遮蔽时长定义为所有导弹都被遮挡的时间段的测度。即问题五的优化模型(14)的目标函数（约束条件不变）需要修改为：

$$\max \quad m \left[\bigcap_{k=1}^3 \bigcup_{i=1}^5 \bigcup_{j=1}^3 \mathcal{T} \circ \mathbf{g}(v_{\text{FY},i}, \alpha_{\text{FY},i}, t_{D_{i,j},\text{thr}}, t_{D_{i,j},\text{bst}}, \mathbf{P}_{\text{FY},i,0}, \mathbf{P}_{\text{Mk},0}) \right]$$

参考文献

- [1] Claude, 4.0 Sonnet, ANTHROPIC, 2025-09-05 至 2025-09-07.
- [2] 豆包, Doubao-Seed-1.6, 字节跳动, 2025-09-05 至 2025-09-07.

附录 A 文件列表

文件名	功能描述
tools.py	模型通用计算代码
prob1.py	问题一程序代码
prob2.py	问题二程序代码
prob3.py	问题三程序代码
prob4.py	问题四程序代码
FY123_prob5.py	问题五 FY1、FY2、FY3 的参数优化程序代码
FY4_prob5.py	问题五 FY4 的参数优化程序代码
FY5_prob5.py	问题五 FY5 的参数优化程序代码
CAL_prob5.py	问题五投弹坐标及起爆坐标计算程序代码
result1.xlsx	问题三结果
result2.xlsx	问题四结果
result3.xlsx	问题五结果
AI 工具使用报告.pdf	AI 使用报告

附录 B 代码

tools.py

```
1 import numpy as np
2 from numpy import array as a
3
4 # 定义重力加速度
5 g = 9.8 # 单位: m/s^2
6 #定义导弹速度
7 V_missile = 300 # 单位: m/s
8 # 真目标的位置坐标
9 point_target = a([0, 200, 0])
10
11 # 定义投弹后且爆炸前/落地前的烟雾弹的位置坐标的函数
12 def CalPos(point, t1, s, v1 = 120, t0 = 1.5):
13     """
```

```

14     points: 导弹/无人机的初始位置坐标
15     v1: 无人机的速度大小
16     t1: 从投弹开始计算的时间
17     t0: 从受领任务开始到投弹的时间
18     """
19     c = point + v1 * t0 * s # 投弹处的位置坐标
20     r = v1 * s * t1 + a([0, 0, -g * t1**2 / 2]) + c
21     return r
22
23 def point_to_line_segment_distance(point, start_point,
24     end_point):
25     """
26     计算空间中一点到线段的距离
27
28     参数:
29     point: 空间中的点 [x, y, z]
30     start_point: 线段的起始点 [x1, y1, z1]
31     end_point: 线段的终止点 [x2, y2, z2]
32
33     返回:
34     距离值
35     """
36     # 将输入转换为numpy数组
37     P = a(point)
38     A = a(start_point)
39     B = a(end_point)
40
41     # 计算线段向量AB
42     AB = B - A
43
44     # 计算向量AP
45     AP = P - A
46
47     # 如果线段长度为0（起点和终点相同），返回点到起点的距离
48     if np.linalg.norm(AB) < 1e-10:

```

```

48         return np.linalg.norm(AP)
49
50     # 计算投影参数t
51     t = np.dot(AP, AB) / np.dot(AB, AB)
52
53     # 根据t值确定最近点的位置
54     if t < 0:
55         # 最近点是线段的起点A
56         closest_point = A
57     elif t > 1:
58         # 最近点是线段的终点B
59         closest_point = B
60     else:
61         # 最近点在线段内部
62         closest_point = A + t * AB
63
64     # 计算点到最近点的距离
65     distance = np.linalg.norm(P - closest_point)
66
67     return distance
68
69 # 定义烟雾弹爆炸后的位置坐标的函数
70 def CalPosAfterBoom(point, t_total, s, v1=120, t0=1.5, t1=3.6)
71     :
72     """
73     计算烟雾弹爆炸后的位置坐标
74
75     参数:
76     point: 无人机的初始位置坐标
77     t_total: 从受领任务开始计算的总时间
78     s: 无人机的飞行方向向量
79     v1: 无人机的速度大小
80     t0: 从受领任务开始到投弹的时间
81     t1: 从投弹后烟雾弹爆炸时间

```



```

82     返回：
83     烟雾弹在t_total时刻的位置坐标
84     """
85     # 如果还没有投弹，返回None
86     if t_total < t0:
87         return None
88
89     # 如果还没有爆炸，使用CalPos计算投弹后爆炸前的位置
90     if t_total < t0 + t1:
91         t_after_drop = t_total - t0 # 从投弹开始的时间
92         return CalPos(point, t_after_drop, s, v1, t0)
93
94     # 爆炸后的情况
95     t2 = t_total - t0 - t1 # 从爆炸开始的时间
96
97     c = point + v1 * t0 * s # 投弹处的位置坐标
98
99     # 计算烟雾弹实际落地的时间（从投弹开始）
100    # 根据  $z = c[2] + v1 * s[2] * t - g * t^2 / 2 = 0$  求解t
101    a = -g / 2
102    b = v1 * s[2]
103    c_coeff = c[2]
104
105    # 解二次方程  $at^2 + bt + c = 0$ 
106    discriminant = b**2 - 4*a*c_coeff
107
108    # 计算落地时间
109    t_land1 = (-b + np.sqrt(discriminant)) / (2*a)
110    t_land2 = (-b - np.sqrt(discriminant)) / (2*a)
111
112    # 选择正的、合理的解
113    t_land = 0
114    for t in [t_land1, t_land2]:
115        if t > 0:
116            t_land = t

```

```

117         break
118
119     if t_land > t1:
120         # 如果烟雾弹在爆炸时间之后才落地，按正常爆炸处理
121         boom_pos = CalPos(point, t1, s, v1, t0)
122         new_z = boom_pos[2] - 3.0 * t2
123         return a([boom_pos[0], boom_pos[1], new_z])
124
125     # 烟雾弹在爆炸前就落地了，计算实际落地位置
126     boom_pos = CalPos(point, t_land, s, v1, t0)
127     # 爆炸后烟雾弹就在这个落地位置保持不变
128     new_z = boom_pos[2] - 3.0 * t2
129     return a([boom_pos[0], boom_pos[1], new_z])
130
131 # 定义计算某个无人机投弹后的烟雾弹对导弹的遮挡时间的函数
132 def timeMissile(point1, point2, s, v1=120, t0=1.5, t1=3.6):
133     """
134     计算某个无人机投弹后的烟雾弹对导弹的遮挡时间
135
136     参数：
137     point1: 无人机的初始位置坐标
138     point2: 导弹的初始位置坐标
139     s: 无人机的飞行方向向量
140     v1: 无人机的速度大小
141     t0: 从受领任务开始到投弹的时间
142     t1: 从投弹后烟雾弹爆炸时间
143
144     返回：
145     (total_blocking_time, start_time, end_time)
146     total_blocking_time: 总遮挡时间（秒）
147     start_time: 开始遮挡时间（从受领任务开始计时，秒）
148     end_time: 结束遮挡时间（从受领任务开始计时，秒）
149     """
150     s2 = -point2 / np.linalg.norm(point2) # 导弹指向的方向向
量

```

```

151
152     # 从爆炸时刻开始检测遮挡
153     t_start_check = t0 + t1 # 爆炸时刻（从受领任务开始计时）
154     t_total = t_start_check # 当前总时间
155     dt = 0.01 # 时间步长
156     total_blocking_time = 0
157     smoke_lifetime = 20.0 # 烟雾弹存在时间20秒
158     t_end_check = t_start_check + smoke_lifetime # 检测结束时间
159
160     start_time = None # 第一次开始遮挡的时间
161     end_time = None # 最后一次结束遮挡的时间
162     currently_blocking = False # 当前是否正在遮挡
163
164     while t_total <= t_end_check:
165         # 计算导弹当前位置（导弹从t=0开始飞行）
166         p_cur = point2 + V_missile * t_total * s2
167
168         # 检查导弹是否已经到达目标
169         if np.linalg.norm(p_cur - point_target) < 1e-2:
170             if currently_blocking:
171                 end_time = t_total
172             break
173
174         # 获取烟雾弹当前位置
175         smoke_pos = CalPosAfterBoom(point1, t_total, s, v1, t0
, t1)
176
177         # 如果烟雾弹位置为None（还未投弹或其他异常），跳过
178         if smoke_pos is None:
179             t_total += dt
180             continue
181
182         # 计算烟雾弹到导弹-目标线段的距离
183         distance = point_to_line_segment_distance(smoke_pos,

```

```

184     p_cur, point_target)
185
186     # 检查遮挡状态
187     is_blocking = distance <= 10
188
189     if is_blocking:
190         total_blocking_time += dt
191         if not currently_blocking:
192             # 开始遮挡
193             if start_time is None:
194                 start_time = t_total
195                 currently_blocking = True
196         else:
197             if currently_blocking:
198                 # 结束遮挡
199                 end_time = t_total
200                 currently_blocking = False
201
202         t_total += dt
203
204     # 如果一直遮挡到最后
205     if currently_blocking and end_time is None:
206         end_time = t_total
207
208     # 如果没有遮挡过
209     if start_time is None:
210         start_time = 0
211         end_time = 0
212
213     return total_blocking_time, start_time, end_time
214
215 def r(t,t0,s,FY=a([17800,0,1800]),v=70):
216     s = a(s)
217     FY = a(FY)
218     g = 9.8
219     return FY + s * t * v - (0.5 * g * (t-t0)**2 * a

```

```

218     ([0,0,1]))
219 def S(theta):
220     return a([np.cos(theta),np.sin(theta),0])

```

probl.py

```

1  import numpy as np
2  from numpy import array as a
3  from tools import *
4  cout = print
5  p1 = a([17800, 0, 1800])
6  M1 = a([20000, 0, 2000])
7
8  total_time, start_time, end_time = timeMissile(p1, M1, a([-1,
9      0, 0]))
10 cout(f"总遮挡时间: {total_time:.2f}秒, 开始遮挡时间: {
    start_time:.2f}秒, 结束遮挡时间: {end_time:.2f}秒")

```

prob2.py

```

1  import numpy as np
2  from numpy import array as a
3  import time
4  import warnings
5  warnings.filterwarnings('ignore')
6  from tools import *
7  cout = print
8
9  def objective_function(params):
10
11     t1, theta = params
12
13     v1 = 70
14     t0 = 0
15     s = a([np.cos(theta), np.sin(theta), 0])
16

```

```

17     p1 = a([17800, 0, 1800])
18     M1 = a([20000, 0, 2000])
19
20     try:
21         total_time, start, end = timeMissile(p1, M1, s, v1, t0
, t1)
22         return total_time
23     except:
24         return 0.0
25
26 def local_grid_search(objective_func, center_point,
search_radius, n_points=25):
27     """在指定点附近进行局部网格搜索"""
28     t1_center, theta_center = center_point
29     t1_radius, theta_radius = search_radius
30
31     t1_range = np.linspace(t1_center - t1_radius, t1_center +
t1_radius, n_points)
32     theta_range = np.linspace(theta_center - theta_radius,
theta_center + theta_radius, n_points)
33
34     best_params = None
35     best_score = -float('inf')
36
37     for t1 in t1_range:
38         for theta in theta_range:
39             score = objective_func([t1, theta])
40             if score > best_score:
41                 best_score = score
42                 best_params = [t1, theta]
43
44     return best_params, best_score
45
46
47 initial_point = [2.467, 3.07]

```

```

48 search_radius = [0.3, 0.1]
49
50 cout("网格搜索优化")
51 cout("="*50)
52
53 start_time = time.time()
54 grid_params, grid_score = local_grid_search(
55     objective_function,
56     initial_point,
57     search_radius,
58     n_points=25
59 )
60 grid_time = time.time() - start_time
61
62 cout(f"最优参数: t1={grid_params[0]:.6f}, theta={grid_params
    [1]:.6f}")
63 cout(f"最大遮挡时间: {grid_score:.6f} 秒")
64 cout(f"计算用时: {grid_time:.2f} 秒")
65
66 # 计算方向向量
67 best_s = a([np.cos(grid_params[1]), np.sin(grid_params[1]),
    0])
68 cout(f"最优方向向量: [{best_s[0]:.6f}, {best_s[1]:.6f}, {
    best_s[2]:.6f}])")
69
70 from tools import *
71 cout(f"投弹点坐标{r(0,t0=0,s=best_s,FY=a([17800,0,1800]),v=70)
    },起爆点坐标{r(grid_params[0],t0=0,s=best_s,FY=a
    ([17800,0,1800]),v=70)}")

```

prob3.py

```

1 import numpy as np
2 from numpy import array as a
3 import time
4 import warnings

```

```

5 warnings.filterwarnings('ignore')
6 from tools import *
7 cout = print
8
9 def objective_function(params):
10     """目标函数：最大化遮挡时间区间的长度"""
11     t0, t1 = params
12
13     # 固定参数
14     v1 = 70
15     s = a([-0.9983, 0.0585, 0])
16
17     p1 = a([17800, 0, 1800])
18     M1 = a([20000, 0, 2000])
19
20     try:
21         total_time, start, end = timeMissile(p1, M1, s, v1, t0
22 , t1)
23
24         # 如果没有遮挡，返回0
25         if total_time <= 0 or start >= end:
26             return 0.0
27
28         # 直接最大化遮挡区间长度
29         return total_time
30
31     except:
32         return 0.0
33
34 def local_grid_search(objective_func, center_point,
35 search_radius, n_points=25):
36     """在指定点附近进行局部网格搜索"""
37     t0_center, t1_center = center_point
38     t0_radius, t1_radius = search_radius

```



```

38     t0_range = np.linspace(t0_center - t0_radius, t0_center +
t0_radius, n_points)
39     t1_range = np.linspace(t1_center - t1_radius, t1_center +
t1_radius, n_points)
40
41     best_params = None
42     best_score = -float('inf')
43
44     for t0 in t0_range:
45         for t1 in t1_range:
46             score = objective_func([t0, t1])
47             if score > best_score:
48                 best_score = score
49                 best_params = [t0, t1]
50
51     return best_params, best_score
52
53
54
55 initial_points = [[3, 3], [6, 3]]
56 search_radiuses = [[2.0, 3], [3.0, 3]]
57
58 for i in range(2):
59     initial_point = initial_points[i]
60     search_radius = search_radiuses[i]
61
62     cout("网格搜索优化（最大化遮挡时间区间长度）")
63     cout("="*60)
64
65     start_time = time.time()
66     grid_params, grid_score = local_grid_search(
67         objective_function,
68         initial_point,
69         search_radius,
70         n_points=50

```

```

71     )
72     grid_time = time.time() - start_time
73
74     cout(f"最优参数: t0={grid_params[0]:.6f}, t1={grid_params
75         [1]:.6f}")
76     cout(f"最大遮挡时间: {grid_score:.6f} 秒")
77     cout(f"计算用时: {grid_time:.2f} 秒")
78
79 # 显示详细信息
80 s = a([-0.9983, 0.0585, 0])
81 v1 = 70
82 cout(f"固定参数: v1={v1}, s=[{s[0]:.4f}, {s[1]:.4f}, {s[2]:.4f
83     }]")
84
85 p1 = a([17800, 0, 1800])
86 M1 = a([20000, 0, 2000])
87
88 total_time, t_start, t_end = timeMissile(p1, M1, s, v1,
89     grid_params[0], grid_params[1])
90
91 cout(f"原始遮挡时间: total_time={total_time:.6f} 秒")
92 cout(f"遮挡区间: [{t_start:.6f}, {t_end:.6f}]")
93
94 from tools import *
95 SET = [[0, 2.467], [1, 2.693878], [2, 0]]
96 for idx, item in enumerate(SET):
97     cout(f"第{idx+1}枚烟幕弹投弹坐标: {r(item[0],item[0], s,
98         p1,v1)}")
99     cout(f"第{idx+1}枚烟幕弹引爆点坐标: {r(item[1] + item[0],
100         item[0], s, p1,v1)}")

```

prob4.py

```

1 import numpy as np
2 from numpy import array as a
3 import time
4 from tools import *

```

```

5 from tqdm import tqdm
6 cout = print
7
8 def optimize_blocking_parameters(point1, point2, param):
9     """
10     使用网格化搜索优化参数以最大化遮挡时间
11
12     参数:
13     point1: 无人机初始位置 [x, y, z]
14     point2: 导弹初始位置 [x, y, z]
15
16     返回:
17     最优参数字典和最佳遮挡结果
18     """
19     # 定义搜索范围
20
21     v1_range, theta_range, t0_range, t1_range = param
22     best_params = None
23     best_blocking_time = 0
24     best_results = None
25
26     total_combinations = len(v1_range) * len(theta_range) *
len(t0_range) * len(t1_range)
27     cout(f"开始网格搜索, 总共 {total_combinations:,} 种组合...")
28
29     start_time = time.time()
30
31     # 创建进度条
32     with tqdm(total=total_combinations,
33               desc="搜索进度",
34               unit="组合",
35               ncols=100,
36               bar_format='{l_bar}{bar}| {n_fmt}/{total_fmt} [{
elapsed}<{remaining}, {rate_fmt}]') as pbar:

```

```

37
38     for v1 in v1_range:
39         for theta in theta_range:
40             # 计算方向向量
41             s = a([np.cos(theta), np.sin(theta), 0])
42
43             for t0 in t0_range:
44                 for t1 in t1_range:
45                     try:
46                         # 计算遮挡时间
47                         total_blocking_time, start_block,
end_block = timeMissile(
48                             point1, point2, s, v1, t0, t1
49                         )
50
51                         # 更新最佳参数
52                         if total_blocking_time >
best_blocking_time:
53                             best_blocking_time =
total_blocking_time
54
55                             best_params = {
56                                 'v1': v1,
57                                 'theta': theta,
58                                 's': s.copy(),
59                                 't0': t0,
60                                 't1': t1
61                             }
62                             best_results = {
63                                 'total_blocking_time':
total_blocking_time,
64                                 'start_time': start_block,
65                                 'end_time': end_block
66                             }
67                             # 更新进度条描述显示当前最佳遮
挡时间

```

```

67         pbar.set_postfix({
68             '最佳遮挡时间': f'{
best_blocking_time:.4f}s',
69             'v1': f'{v1:.1f}',
70             'θ': f'{np.degrees(theta)
:.1f}°'
71         })
72
73     except Exception as e:
74         pass
75
76     # 更新进度条
77     pbar.update(1)
78
79     elapsed_time = time.time() - start_time
80     cout(f"\n搜索完成! 总耗时: {elapsed_time:.1f}秒")
81
82     # 打印结果
83     if best_params:
84         cout("\n" + "="*50)
85         cout("最优参数:")
86         cout("="*50)
87         cout(f"v1 = {best_params['v1']:.2f} m/s")
88         cout(f"theta = {best_params['theta']:.4f} 弧度 ({np.
degrees(best_params['theta']):.1f}°)")
89         cout(f"s = [{best_params['s'][0]:.4f}, {best_params['s
'][1]:.4f}, {best_params['s'][2]:.4f}]")
90         cout(f"t0 = {best_params['t0']:.2f} 秒")
91         cout(f"t1 = {best_params['t1']:.2f} 秒")
92         cout(f"\n最大遮挡时间: {best_results['
total_blocking_time']:.4f} 秒")
93         cout(f"遮挡开始时间: {best_results['start_time']:.2f}
秒")
94         cout(f"遮挡结束时间: {best_results['end_time']:.2f} 秒
")

```

```

95         cout("="*50)
96     else:
97         cout("未找到有效参数组合")
98     return best_params, best_results
99
100
101 # # 手动设置无人机和导弹的初始位置
102 FY2 = a([12000, 1400, 1400]) # FY2初始位置
103 FY3 = a([6000, -3000, 700]) # FY3初始位置
104 M1 = a([20000, 0, 2000]) # 导弹初始位置
105 # FY2搜索范围
106 param1 = (np.linspace(124, 125, 10), np.linspace(4, 5, 5), np.
    linspace(5, 6, 10), np.linspace(6, 7, 10))
107 # FY3搜索范围
108 param2 = (np.linspace(105, 107, 10), np.linspace(1.8, 2, 5),
    np.linspace(23, 25, 10), np.linspace(5, 7, 10))
109 cout("FY2最优参数: ")
110 best_params, best_results = optimize_blocking_parameters(FY2,
    M1, param1)
111 cout("FY3最优参数: ")
112 best_params, best_results = optimize_blocking_parameters(FY3,
    M1, param2)
113
114
115 SET = [[5, 6, S(4.75), FY2, 124], [24.33, 5.67, S(1.7778), FY3,
    105.44]]
116 for i in range(2):
117     cout(f"第{i+2}个无人机投弹坐标: {r(SET[i][0], SET[i][0],
    SET[i][2], SET[i][3], SET[i][4])}")
118     cout(f"第{i+2}个无人机烟幕弹引爆点坐标: {r(SET[i][1] + SET
    [i][0], SET[i][0], SET[i][2], SET[i][3], SET[i][4])}")

```

FY123_prob5.py

```

1 import numpy as np
2 from numpy import array as a

```

```

3 import time
4 import warnings
5 warnings.filterwarnings('ignore')
6 from prob1 import *
7 cout = print
8
9 def objective_function(params):
10     """目标函数：最大化遮挡时间区间的长度"""
11     t0, t1, theta, v1, p, M= params
12     s = a([np.cos(theta), np.sin(theta), 0])
13     try:
14         total_time, start, end = timeMissile(p, M, s, v1, t0,
15         t1)
16         # 如果没有遮挡，返回0
17         if total_time <= 0 or start >= end:
18             return 0.0
19         # 直接最大化遮挡区间长度
20         return total_time
21     except:
22         return 0.0
23
24 def local_grid_search(objective_func, center_point,
25 search_radius, n_points=25, other_params=None):
26     """在指定点附近进行局部网格搜索"""
27     theta, v1, p, M = other_params
28     t0_center, t1_center = center_point
29     t0_radius, t1_radius = search_radius
30
31     t0_range = np.linspace(t0_center - t0_radius, t0_center +
32 t0_radius, n_points)
33     t1_range = np.linspace(t1_center - t1_radius, t1_center +
34 t1_radius, n_points)
35
36     best_params = None
37     best_score = -float('inf')

```

```

34
35     for t0 in t0_range:
36         for t1 in t1_range:
37             score = objective_func([t0, t1, theta, v1, p, M])
38             if score > best_score:
39                 best_score = score
40                 best_params = [t0, t1]
41
42     return best_params, best_score
43
44
45 FY1 = a([17800, 0, 1800])
46 FY2 = a([12000, 1400, 1400])
47 FY3 = a([6000, -3000, 700])
48 FY4 = a([11000, 2000, 1800])
49 FY5 = a([13000, -2000, 1300])
50 M1 = a([20000, 0, 2000])
51 M2 = a([19000, 600, 2100])
52 M3 = a([18000, -600, 1900])
53
54 # 初始点设置: [t0, t1]
55 # FY2遮挡M2参数搜索
56 initial_point = [4, 3.8] #每次搜索更换这个参数
57 search_radius = [0.3, 0.3]
58 cout("网格搜索优化 (最大化遮挡时间区间长度)")
59 cout("="*60)
60 start_time = time.time()
61 grid_params, grid_score = local_grid_search(
62     objective_function,
63     initial_point,
64     search_radius,
65     n_points=50,
66     other_params=[4.75, 124, FY2, M2]
67 )
68 grid_time = time.time() - start_time

```



```

69 cout(f"最优参数: t0={grid_params[0]:.6f}, t1={grid_params
    [1]:.6f}")
70 cout(f"最大遮挡时间: {grid_score:.6f} 秒")
71 cout(f"计算用时: {grid_time:.2f} 秒")
72
73 # FY2遮挡M3参数搜索
74 initial_point = [9.5, 5] #每次搜索更换这个参数
75 search_radius = [0.3, 0.3]
76 cout("网格搜索优化 (最大化遮挡时间区间长度)")
77 cout("="*60)
78 start_time = time.time()
79 grid_params, grid_score = local_grid_search(
80     objective_function,
81     initial_point,
82     search_radius,
83     n_points=50,
84     other_params=[4.75, 124, FY2, M3]
85 )
86 grid_time = time.time() - start_time
87 cout(f"最优参数: t0={grid_params[0]:.6f}, t1={grid_params
    [1]:.6f}")
88 cout(f"最大遮挡时间: {grid_score:.6f} 秒")
89 cout(f"计算用时: {grid_time:.2f} 秒")
90
91 # FY3遮挡M3参数搜索
92 initial_point = [23, 5] #每次搜索更换这个参数
93 search_radius = [0.3, 0.3]
94 cout("网格搜索优化 (最大化遮挡时间区间长度)")
95 cout("="*60)
96 start_time = time.time()
97 grid_params, grid_score = local_grid_search(
98     objective_function,
99     initial_point,
100     search_radius,
101     n_points=50,

```

```

102     other_params=[1.7778, 105.44, FY3, M3]
103 )
104 grid_time = time.time() - start_time
105 cout(f"最优参数: t0={grid_params[0]:.6f}, t1={grid_params
    [1]:.6f}")
106 cout(f"最大遮挡时间: {grid_score:.6f} 秒")
107 cout(f"计算用时: {grid_time:.2f} 秒")
108
109
110 # FY3遮挡M2参数搜索
111 initial_point = [27, 5] #每次搜索更换这个参数
112 search_radius = [0.3, 0.5]
113 cout("网格搜索优化 (最大化遮挡时间区间长度)")
114 cout("="*60)
115 start_time = time.time()
116 grid_params, grid_score = local_grid_search(
117     objective_function,
118     initial_point,
119     search_radius,
120     n_points=50,
121     other_params=[1.7778, 105.44, FY3, M2]
122 )
123 grid_time = time.time() - start_time
124 cout(f"最优参数: t0={grid_params[0]:.6f}, t1={grid_params
    [1]:.6f}")
125 cout(f"最大遮挡时间: {grid_score:.6f} 秒")
126 cout(f"计算用时: {grid_time:.2f} 秒")

```

FY4_prob5.py

```

1 import numpy as np
2 from numpy import array as a
3 from tools import *
4 from sko.GA import GA
5 cout = print
6

```

```

7 def obj_func(params):
8     """目标函数：最大化导弹运行时间"""
9     # 解包参数
10    theta, v1, t0, t1 = params
11    # 定义相关数组
12    FY4 = a([11000, 2000, 1800])
13
14    M2 = a([19000, 600, 2100])
15    s = a([np.cos(theta), np.sin(theta), 0])
16
17    # 计算时间
18    total_time, _, _ = timeMissile(FY4, M2, s, v1, t0, t1)
19
20    # 返回负值，因为遗传算法默认寻找最小值
21    return -total_time
22
23    cout("注意，中间过程可能会由于SKO库版本的缘故输出一些与结果无
        关的参数与图片，可以忽略。")
24    cout("为避免混淆结果，在输出真正的参数之前会输出一行#号以作区
        分")
25
26    # 创建遗传算法实例
27    ga = GA(
28        func=obj_func,          # 目标函数
29        n_dim=4,                 # 变量维度 (theta, v1, t0, t1)
30        size_pop=500,           # 种群大小 (每代个体数量)
31        max_iter=100,           # 最大迭代次数
32        probab_mut=0.1,         # 变异概率
33        lb=[4.6, 120, 0, 0],    # 变量下界 [theta_min, v1_min,
                                # t0_min, t1_min]
34        ub=[5.2, 140, 20, 10], # 变量上界 [theta_max, v1_max,
                                # t0_max, t1_max]
35        precision=1e-3          # 变量精度
36    )
37
38    # 运行遗传算法
39    best_params, best_value = ga.run()

```

```

38 # 输出结果
39 cout('#'*50)
40 cout('最优参数组合:', best_params)
41 cout('最优时间值:', -best_value) # 转换回正值
42
43 # =====#
44 # 搜索FY5对M1和M3遮挡的最优参数组合#
45 FY4 = a([11000, 2000, 1800])
46 M1 = a([20000, 0, 2000])
47 M3 = a([18000, -600, 1900])
48
49 from FY123_prob5 import objective_function, local_grid_search
50 # 初始点设置: [t0, t1]
51 # FY4遮挡M1参数搜索
52 initial_point = [6, 5] #每次搜索更换这个参数
53 search_radius = [2.14, 5]
54 cout("网格搜索优化 (最大化遮挡时间区间长度)")
55 cout("="*60)
56
57 grid_params, grid_score = local_grid_search(
58     objective_function,
59     initial_point,
60     search_radius,
61     n_points=50,
62     other_params=[5.16363636, 138.18, FY4, M1]
63 )
64 cout(f"最优参数: t0={grid_params[0]:.6f}, t1={grid_params
65     [1]:.6f}")
66
67
68 # FY4遮挡M3参数搜索
69 initial_point = [8, 5] #每次搜索更换这个参数
70 search_radius = [3.14, 5]
71 cout("网格搜索优化 (最大化遮挡时间区间长度)")

```

```

72 cout("="*60)
73
74 grid_params, grid_score = local_grid_search(
75     objective_function,
76     initial_point,
77     search_radius,
78     n_points=50,
79     other_params=[5.16363636, 138.18, FY4, M3]
80 )
81 cout(f"最优参数: t0={grid_params[0]:.6f}, t1={grid_params
    [1]:.6f}")
82 cout(f"最大遮挡时间: {grid_score:.6f} 秒")

```

FY5_prob5.py

```

1  import numpy as np
2  from numpy import array as a
3  from tools import *
4  from sko.GA import GA
5  cout = print
6
7  def obj_func(params):
8      """目标函数：最大化导弹运行时间"""
9      # 解包参数
10     theta, v1, t0, t1 = params
11
12     # 定义相关数组
13     FY5 = a([13000, -2000, 1300])
14     M1 = a([20000, 0, 2000])
15     s = a([np.cos(theta), np.sin(theta), 0])
16
17     # 计算时间
18     total_time, _, _ = timeMissile(FY5, M1, s, v1, t0, t1)
19
20     # 返回负值，因为遗传算法默认寻找最小值
21     return -total_time

```

```

22 cout("注意，中间过程可能会由于SKO库版本的缘故输出一些与结果无
    关的参数与图片，可以忽略。")
23 cout("为避免混淆结果，在输出真正的参数之前会输出一行#号以作区
    分")
24 # 创建遗传算法实例
25 ga = GA(
26     func=obj_func,          # 目标函数
27     n_dim=4,                # 变量维度 (theta, v1, t0, t1)
28     size_pop=500,           # 种群大小 (每代个体数量)
29     max_iter=100,           # 最大迭代次数
30     probab_mut=0.1,         # 变异概率
31     lb=[1.2, 120, 0, 0],    # 变量下界 [theta_min, v1_min,
    t0_min, t1_min]
32     ub=[2.2, 140, 20, 10], # 变量上界 [theta_max, v1_max,
    t0_max, t1_max]
33     precision=1e-3          # 变量精度
34 )
35
36 # 运行遗传算法
37 best_params, best_value = ga.run()
38 # 输出结果
39 cout('#'*50)
40 cout('最优参数组合:', best_params)
41 cout('最优时间值:', -best_value) # 转换回正值
42
43 # =====#
44 # 搜索FY5对M1和M3遮挡的最优参数组合#
45 FY5 = a([13000, -2000, 1300])
46 M2 = a([19000, 600, 2100])
47 M3 = a([18000, -600, 1900])
48
49 from FY123_prob5 import objective_function, local_grid_search
50 # 初始点设置: [t0, t1]
51 # FY5遮挡M3参数搜索
52 initial_point = [6.4, 5] #每次搜索更换这个参数

```

```

53 search_radius = [6.4, 5]
54 cout("网格搜索优化 (最大化遮挡时间区间长度)")
55 cout("="*60)
56
57 grid_params, grid_score = local_grid_search(
58     objective_function,
59     initial_point,
60     search_radius,
61     n_points=50,
62     other_params=[1.617778, 140, FY5, M3]
63 )
64 cout(f"最优参数: t0={grid_params[0]:.6f}, t1={grid_params
    [1]:.6f}")
65 cout(f"最大遮挡时间: {grid_score:.6f} 秒")
66
67
68 # FY5 遮挡M2参数搜索
69 initial_point = [17, 5] #每次搜索更换这个参数
70 search_radius = [2.14, 5]
71 cout("网格搜索优化 (最大化遮挡时间区间长度)")
72 cout("="*60)
73
74 grid_params, grid_score = local_grid_search(
75     objective_function,
76     initial_point,
77     search_radius,
78     n_points=50,
79     other_params=[1.617778, 140, FY5, M2]
80 )
81 cout(f"最优参数: t0={grid_params[0]:.6f}, t1={grid_params
    [1]:.6f}")
82 cout(f"最大遮挡时间: {grid_score:.6f} 秒")

```

CAL_prob5.py

```

1 from tools import *

```

```

2 from numpy import array as a
3 cout = print
4 FY1 = a([17800, 0, 1800])
5 FY2 = a([12000, 1400, 1400])
6 FY3 = a([6000, -3000, 700])
7 FY4 = a([11000, 2000, 1800])
8 FY5 = a([13000, -2000, 1300])
9 item = [[[0, 2.467, S(3.083),FY1, 70], [1, 2.693878,S(3.083),
    FY1, 70], [2, 0, S(3.083),FY1, 70]],
10         [[4.275510, 3.561224, S(4.75), FY2, 124], [5, 6, S
    (4.75), FY2, 124], [9.461633, 4.956122, S(4.75), FY2, 124]],
11         [[23.041633, 5.058367, S(1.7778), FY3, 105.44],
    [24.33, 5.67, S(1.7778), FY3, 105.44], [26.809388, 4.672857,
    S(1.7778), FY3, 105.44]],
12         [[2.86, 9.99, S(5.16363636), FY4, 138.18], [3.86 +
    9.99, 0, S(5.16363636), FY4, 138.18], [4.86 + 9.99, 0, S
    (5.16363636), FY4, 138.18]],
13         [[12.86131966, 0, S(1.617778),FY5, 140],[13.86131966,
    0.63736264, S(1.617778),FY5, 140], [14.86131966, 0, S
    (1.617778),FY5, 140]]]
14
15 for i in range(len(item)):
16     for j in range(len(item[i])):
17         cout(f"第{i+1}个无人机投{j+1}个弹的投弹坐标: {r(item[i]
    ][j][0], item[i][j][0], item[i][j][2], item[i][j][3], item[i]
    ][j][4])}")
18         cout(f"第{i+1}个无人机投{j+1}个弹的起爆坐标: {r(item[i]
    ][j][0] + item[i][j][1], item[i][j][0], item[i][j][2], item[
    i][j][3], item[i][j][4])}")

```